

Генерация перестановок, сочетаний,
размещений без повторений. Генерация
перестановок, сочетаний, размещений с
повторениями. Генерация комбинаций и
разбиений

Комбинаторика

Комбинаторика — это область математики, прежде всего связанная с подсчетом, как средство и цель получения результатов, так и с определением свойств конечных структур.

- Треугольник Паскаля, бином Ньютона
- Правило сложения, правило умножения
- Комбинаторные тождества
- Сочетания
- Размещения
- Перестановки
- Разбиения
- Битовые последовательности

Правило сложения

Пусть есть два множества объектов $A = \{a_1, \dots, a_n\}$ и $B = \{b_1, \dots, b_m\}$, тогда количество способов выбрать один объект из A или один объект из B равно $n + m$.

Пример

В меню столовой колледжа ^A2 вида холодных закусок, ^B5 вариантов первых блюд и ^C7 вариантов вторых. Сколькими способами можно выбрать ОДНО БЛЮДО?

Решение

В задаче рассматриваются три группы: холодные закуски, первые и вторые блюда. Сколько элементов в группах? Закуску можно выбрать 2 способами; первое блюдо выбрать 5 способами; второе блюдо можно выбрать 7 способами. Видим, что в группах нет одинаковых элементов.

Применим закон сложения: $2+5+7=14$.

Ответ: блюдо можно выбрать 14 способами.

Правило умножения

Пусть есть два множества объектов $A = \{a_1, \dots, a_n\}$ и $B = \{b_1, \dots, b_m\}$, тогда количество способов выбрать сперва один объект из A , а затем выбрать один объект из B равно nm .

Пример

На витрине кафе представлено 15 вариантов десертов. Ребята из них выбирают 3 десерта. Выясните, сколькими различными способами можно выбрать 3 десерта?

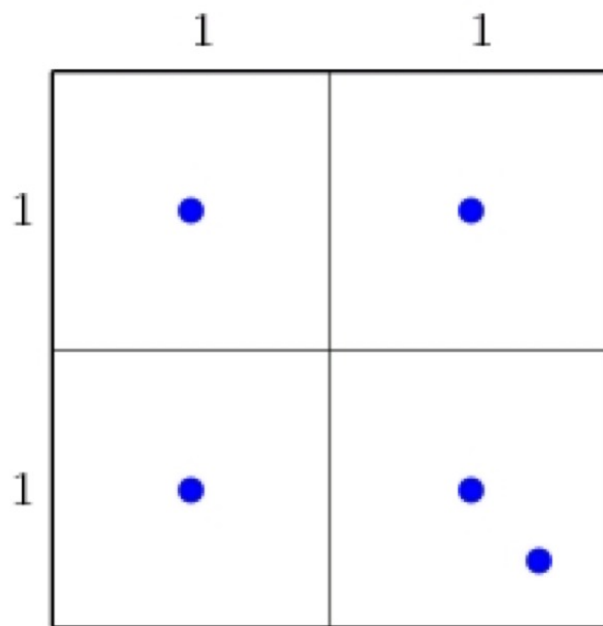
Решение

Сначала ребята могут выбрать любой из всех 15 десертов. Когда первый выбор сделан, для следующего остаётся $15-1=14$ вариантов выбора десерта. 1-й десерт выбирают 15 способами, 2-й десерт выбираем 14 способами. Используем правило произведения: 2 десерта выбираем $15 \cdot 14 = 210$ (способами).

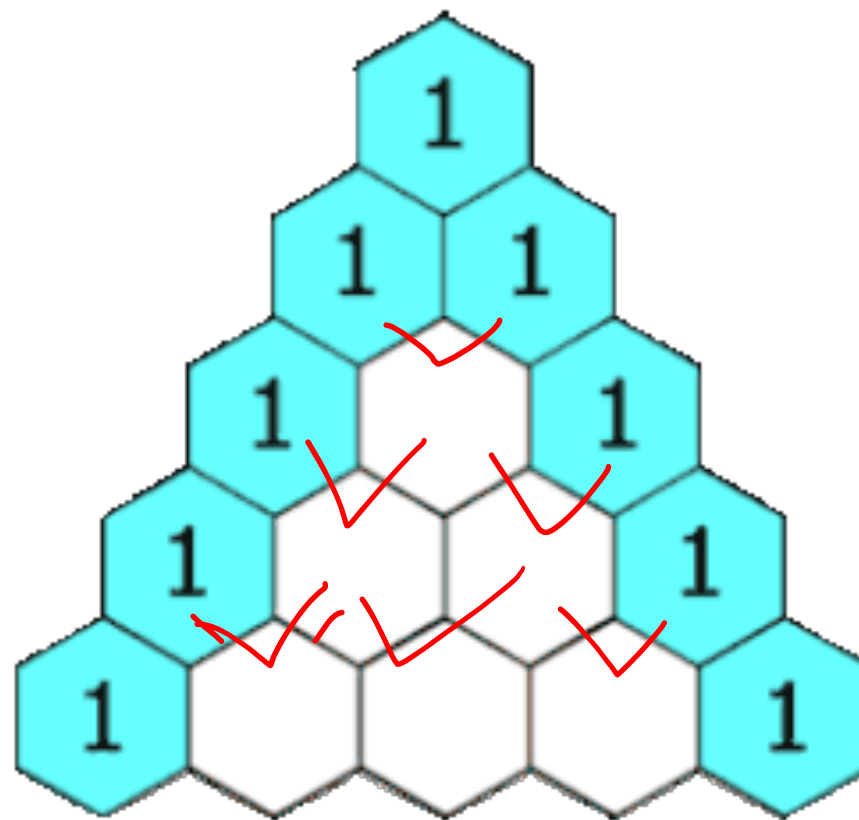
Ответ: Ребята могут выбрать десерт 210 различными способами.

Принцип Дирихле

Пусть есть n ящиков и $n+1$ кролик. Если рассадить всех кроликов по этим ящикам, то обязательно найдется ящик в котором не меньше двух кроликов.



Треугольник Паскаля



Бином Ньютона

Для степеней сумм мы можем расписать формулы

$$(x + y)^1 = x + y$$

$$(x + y)^2 = x^2 + y^2 + 2xy$$

$$(x + y)^3 = x^3 + y^3 + 3x^2y + 3xy^2$$

$$(x + y)^4 = x^4 + y^4 + 4x^3y + 4xy^3 + 6x^2y^2$$

| Каким образом мы можем получить общую формулу для коэффициентов в разложении $(x + y)^n$?

Бином Ньютона

$$(x + y)^n = \sum_{k=0}^n C_n^k x^k y^{n-k}$$

$$x^n + \underbrace{x^{n-1} \cdot y^1}_n + \underbrace{x^{n-2} \cdot y^2}_2 + \dots + y^n$$

Доказательство

Очевидно,

$$(x + y)^n = \underbrace{(x + y)(x + y) \dots (x + y)}$$

и из каждой скобки мы должны взять или x или y . Пусть из k скобок взяли x , тогда из $n - k$ скобок взяли y . Когда перемножим эти переменные, получим слагаемое вида

$$x^k y^{n-k}$$

Сколько таких слагаемых будет? А ровно столько сколько раз мы возьмем конкретные k скобок (из которых возьмем x) из n возможных. Это можно сделать C_n^k числом способов. Получаем, что в разложение $(x + y)^n$ входит слагаемое $C_n^k x^k y^{n-k}$ при конкретном k . Суммируя по k получим требуемое.

Бином Ньютона

Пример

Напишите разложение выражения по формуле бинома Ньютона.

Решение

Смотрим на строку треугольника Паскаля, соответствующую пятой степени. Биномиальными коэффициентами будут числа 1, 5, 10, 10, 5, 1. Таким образом, имеем

$$\underline{(a + b)^5} = a^5 + 5a^4b + 10a^3b^2 + 10a^2b^3 + 5ab^4 + b^5$$

$C_5^0 a^5 \dots C_5^1 a^4 b \dots C_5^2 a^3 b^2 \dots$

Бином Ньютона

Пример

Найдите коэффициент бинома Ньютона для шестого члена разложения выражения $(a + b)^{10}$

$$a^{10} + a^9b^1 + \dots + \boxed{a^5b^5} \\ C_{10}^5$$

Решение

В нашем примере $n=10$, $k=6-1=5$. Таким образом, мы можем вычислить требуемый биномиальный коэффициент:

$$C_n^k = C_{10}^5 = \frac{(10!)}{\underbrace{5! (10 - 5)!}} = \frac{(10!)}{5! 5!} = 252$$

Бином Ньютона

Пример

Доказать, что значение выражения $5^n + 28n - 1$, где n - натуральное число, делится на 16 без остатка.

Бином Ньютона

$$C_n^k = \frac{n!}{k!(n-k)!} \quad C_n^n = \frac{n!}{n! \cdot 0!} = 1$$
$$C_n^{n-1} = \frac{n!}{(n-1)! \cdot 1} = n$$

Пример

Доказать, что значение выражения $5^n + 28n - 1$, где n - натуральное число, делится на 16 без остатка.

Решение

Представим первое слагаемое выражения как $5^n = (4 + 1)^n$ и воспользуемся формулой бинома Ньютона:

$$\begin{aligned} 5^n + 28n - 1 &= (4 + 1)^n + 28n - 1 \\ &= 4^n C_n^0 + 1 \cdot 4^{n-1} C_n^1 + \dots + 4^2 1^{n-2} C_n^{n-2} + 4 \cdot 1^{n-1} C_n^{n-1} + 1^n C_n^n + 28n - 1 \\ &= 4^n + 4^{n-1} C_n^1 + \dots + 4^2 C_n^{n-2} + 4n + 1 + 28n - 1 \\ &= 4^n + 4^{n-1} C_n^1 + \dots + 4^2 C_n^{n-2} + 32n = 16(4^{n-2} + 4^{n-3} C_n^1 + \dots + C_n^{n-2} + 2n) \end{aligned}$$

Полученное произведение доказывает делимость исходного выражения на 16.

Комбинаторные тождества

$$C_n^k = C_n^{n-k}$$

Доказательство

Расписав по формуле

$$C_n^k = \frac{n!}{k! (n-k)!} = \frac{n!}{(n-k)! k!}$$

правую и левую часть рассматриваемого равенства, получим верное тождество.

Комбинаторные тождества

$$C_n^k = C_{n-1}^k + C_{n-1}^{k-1}$$

Доказательство

Нарисуем треугольник Паскаля (это бесконечная таблица коэффициентов, имеющая треугольную форму. В этом треугольнике на вершине и по бокам стоят единицы. Каждое число равно сумме двух расположенных над ним чисел. Строки треугольника симметричны относительно вертикальной оси.)

				1			
			1		1		
		1		2		1	
	1		3		3		1
	1	4		6		4	1
1	5	10		10		5	1

Комбинаторные тождества

Бесконечная таблица биномиальных коэффициентов, имеющая треугольную форму, выглядит следующим образом:

$$\begin{array}{ccccccc} & & & & c_0^0 & & \\ & & & & & & \\ & & & & c_1^0 & & c_1^1 \\ & & & & & & \\ & & & & c_2^0 & & c_2^1 & & c_2^2 \\ & & & & & & \\ & & & & c_3^0 & & c_3^1 & & c_3^2 & & c_3^3 \\ & & & & & & \\ & & & & c_4^0 & & c_4^1 & & c_4^2 & & c_4^3 & & c_4^4 \end{array}$$

Обоснование совпадения этих двух таблиц основано на тождестве 2, которое мы сейчас должны доказать.

Комбинаторные тождества

Рассмотрим правую часть требуемого равенства

$$\begin{aligned} C_{n-1}^k + C_{n-1}^{k-1} &= \frac{(n-1)!}{k!(n-1-k)!} + \frac{(n-1)!}{(k-1)!(n-k)!} \\ &= \frac{(n-1)!(n-k) + (n-1)!k}{k!(n-k)!} = \frac{n!}{k!(n-k)!} = C_n^k \end{aligned}$$

Комбинаторные тождества

$$\sum_{k=0}^n C_n^k = 2^n$$

Доказательство

Используя бином Ньютона, получим

$$2^n = (1 + 1)^n = \sum_{k=0}^n C_n^k \underbrace{1^k 1^{n-k}}_{=1} = \sum_{k=0}^n C_n^k$$

Комбинаторные тождества

$$(C_n^0)^2 + (C_n^1)^2 + \dots + (C_n^n)^2 = C_{2n}^n$$

Доказательство

Просуммируем квадраты значений в строках треугольника Паскаля

			1				$\rightarrow 1 = C_0^0$
			1		1		$\rightarrow 2 = C_2^1$
		1		2		1	$\rightarrow 6 = C_4^2$
	1		3		3		$\rightarrow 20 = C_6^3$
1		4		6		4	$\rightarrow 70 = C_8^4$

Полиномиальная формула

Пример

Рассмотрим слово КОМБИНАТОРИКА и будем переставлять буквы в этом слове. Сколько математических слов в этом случае получится?

Решение

Перечислим все буквы указанного слова:

К - 2, О - 2, М - 1, Б - 1, И - 2, Н - 1, А - 2, Т - 1, Р - 1,

и представим, что мы набираем эти буквы в пустой вектор длины 13.

Сперва выберем 2 места для буквы К, это можно сделать C_{13}^2 числом способов.

Затем выберем 2 места для буквы О, это можно сделать C_{11}^2 числом способов.

Дальнейший выбор мест для букв мы производим $C_9^1, C_8^1, C_7^2, C_5^1, C_4^2, C_2^1, C_1^1$ числом способов. Тогда общее действие состоящее в создании слова может быть выполнено $C_{13}^2 C_{11}^2 C_9^1 C_8^1 C_7^2 C_5^1 C_4^2 C_2^1 C_1^1$ числом способов. Расписав число сочетаний по формуле, получим верный ответ $C_{13}^2 C_{11}^2 C_9^1 C_8^1 C_7^2 C_5^1 C_4^2 C_2^1 C_1^1 = \frac{13!}{2!2!2!}$

Полиномиальная формула

Пусть у нас есть набор символов $a_1, \dots, a_k$, где объектов a_1 — n_1 штук, объектов a_2 — n_2 штук, и число n

$= n_1 + \dots + n_k$. $P(n_1, \dots, n_k)$

— количество способов упорядочить наши n объектов и называется полиномиальный коэффициент.

Рассуждая как в задаче со словом КОМБИНАТОРИКА получим, что верна следующая теорема

$$P(n_1, \dots, n_k) = C_n^{n_1} C_{n-n_1}^{n_2} \dots C_{n-n_1-\dots-n_{k-1}}^{n_k} = \frac{n!}{n_1! n_2! \dots n_k!}$$

Примеры

Пример 1: Сколько слов можно составить, переставив буквы в слове «математика»?

Решение: «математика» - 10 букв (с повт. $m=2, a=3, t=2, e=i=k=1$), $=1$),

Ответ: 151200 слов.

Пример 2: Сколько различных четырёхзначных чисел можно составить из цифр 0, 1, 2, 3, и 4, причём в

каждом числе цифры должны быть разные?

Решение: $P_5 - P_4 = 5! - 4! = 120 - 24 = 96$.

Ответ: 96 чисел.

Формула включений и исключений

Пример

Пусть у нас есть 50 человек, среди которых кто-то может знать английский, кто-то французский и кто-то немецкий язык (могут быть люди которые знают сразу три языка, могут быть те кто знают только один язык). Обозначим через $N(a)$ - количество людей знающих английский язык (возможно, они знают и еще какой-то язык, помимо английского), $N(\phi)$ - количество людей знающих французский язык, $N(n)$ - количество людей знающих немецкий язык. Так же, пусть нам известны $N(a, \phi)$, $N(a, n)$ и $N(\phi, n)$ количества людей знающих пару языков, и известно количество людей знающих все три языка сразу - $N(a, \phi, n)$. Как посчитать количество людей, которые не знают ни один из указанных языков? _____

Решение

Использовать формулу включений исключений.

$$\begin{aligned} & N(a'_1, a'_2, \dots, a'_n) \\ & = N - N(a_1) - \dots - N(a_n) + N(a_1, a_2) + \dots + N(a_{n-1}, a_n) + \dots + (-1)^n N(a_1, a_2, \dots, a_n) \end{aligned}$$

Ответ: $50 - N(a) - N(\phi) - N(n) + N(a, \phi) + N(a, n) + N(\phi, n) - N(a, \phi, n)$.

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$



Сочетания, размещения и перестановки

- 1) число k -размещений без повторений из n объектов обозначается A_n^k
- 2) число k -размещений с повторениями из n объектов обозначается $\overline{A_n^k}$
- 3) число k -сочетаний без повторений из n объектов обозначается C_n^k
(читается „цэ из п по к“, это обозначение придумал Паскаль в 17 веке, но сейчас в англоязычном мире принято обозначение $\binom{n}{k} = C_n^k$)
- 4) число k -сочетаний с повторениями из n объектов обозначается $\overline{C_n^k}$
- 5) число перестановок без повторений длиной n обозначают P_n

Размещения с повторениями

Даны предметы N типов. Из них составляют расстановки длины k . В расстановки могут входить предметы одного типа. Две расстановки считаются различными, если они отличаются или типом входящих в них предметов, или порядком предметов

Пример: машинное слово состоит из $k=16$ бит. В каждом бите можно записать одно из $N=2$ значений 0 или 1. Сколько различных комбинаций 1 и 0 может быть записано в слове?

Ответ: $2^{16} = 65536$

Общее количество размещений с повторениями

$$\overline{A_N^k} = N^k$$

Теорема о количестве размещений с повторениями

$$\overline{A_n^k} = n^k$$

Доказательство

Возьмем n объектов $\{a_1, \dots, a_n\}$. Мы хотим выбирать объект из этого множества и возвращать обратно и так k раз. Это действие можно представить как последовательный выбор объектов по одному из множеств

$A_1 = \{a_1, \dots, a_n\}, A_2 = \{a_1, \dots, a_n\}, \dots, A_k = \{a_1, \dots, a_n\}$.

В силу правила умножения мы получим количество способов выбрать k объектов равным $n \cdot \dots \cdot n = n^k$

Размещения без повторений

Пусть имеется множество S из N элементов, и задано $M \leq N$. **Размещением без повторений из N по M** называют последовательность длины M , составленную из различных элементов S .

Пример 1: В чемпионате участвуют 10 команд. Сколько способов размещения команд на пьедестале (1 золото, 1 серебро, 1 бронза)?

Пример 2: В классе 23 ученика. Сколько способов выбора старосты, культорга, спортсектора?

Каждый элемент можно использовать только один раз, но не требуется использовать все элементы.



Теорема

$$A_n^k = n(n - 1) \dots (n - k + 1) = \frac{n!}{(n - k)!}$$

Доказательство

Выберем из $A = \{a_1, \dots, a_n\}$ первый объект n способами, назад его не возвращаем, затем извлечем из оставшихся $n - 1$ объекта второй объект $n - 1$ способом, и т.д. вплоть до k -го объекта который мы извлечем $n - k + 1$ числом способов. Применим правило умножения и получим, что действие состоящее в извлечении упорядочено без возвращения k объектов из n можно выполнить $n \cdot (n - 1) \dots \cdot (n - k + 1)$ числом способов.

Размещения без повторений

Пример

Для прохождения практики студентов есть 14 автомобильных сервисов. Сколькими способами можно устроить трёх человек, чтобы они были в разных сервисах?

Решение

Требуемая выборка - размещение, т.к. порядок элементов важен. Например, если первый человек будет работать в сервисе А, второй - в В, а третий - в С. Меняя местами людей, получатся новые ситуации - новые выборки. Нужно вычислить, сколькими способами можно выбрать t элементов из n элементов, где $n=14$; $t=3$. Применяя формулу, получаем

$$A_{11}^{14} = 2184$$

Ответ: 2184 способ.

Сочетания без повторений

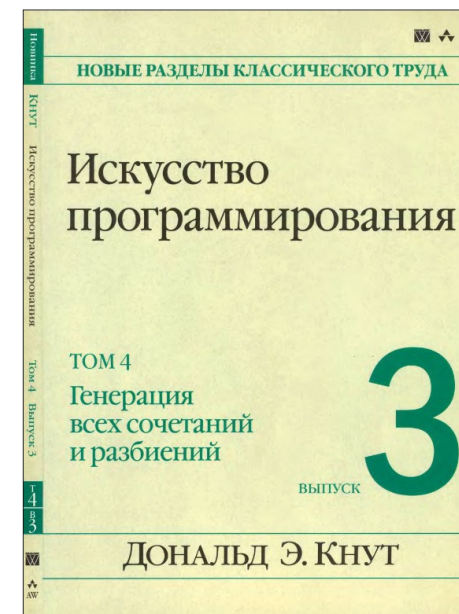
Сочетанием из N элементов по M называют M -элементные подмножества какого-либо N -элементного множества.

Пример, для множества $S = \{a, b, c, d\}$ сочетания из двух элементов (двухэлементные подмножества) $\{a, b\}, \{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{c, d\}$.
Всего $4!/2!/(4-2)! = 1*2*3*4/(1*2)/(1*2) = 2*3 = 6$ сочетаний.

Пример 1. Сколькими способами можно выбрать 2 из 3 монет (1 руб, 5 руб., 10 руб.)?

Пример 2: В чемпионате участвуют 10 команд. Три команды-победители участвуют в Европейских соревнованиях. Сколько вариантов команд - победителей?

Пример 3: В классе 23 ученика. Староста, культорг и спортсектор входят в Совет школы. Сколько способов выбора участников Совета школы из класса?



Теорема о количестве сочетаний без повторений

$$C_n^k = \frac{A_n^k}{k!} = \frac{n!}{(n-k)! k!}$$

Доказательство

Из исходного набора объектов $\{a_1, \dots, a_n\}$ образуем k штук сочетаний: первое сочетание – $\{a_1, \dots, a_k\}$, и т.д., последнее сочетание – $\{a_{n-k+1}, \dots, a_n\}$. Номер последнего сочетания по определению есть C_n^k . Затем мы каждое из полученных сочетаний упорядочиваем $k!$ числом способов и получаем размещения. Следовательно, число размещений – это с одной стороны A_n^k , а с другой стороны $C_n^k k!$. По этому мы получим утверждение нашей теоремы.

Сочетания с повторениями

Надо купить $N=7$ пончиков. Всего видов пончиков $k=4$.

Сколько есть способов набрать 7 пончиков 4 видов?

Закодируем набор с помощью последовательности из нулей и единиц.

Запишем столько единиц, сколько пончиков первого типа, затем нуль, затем столько единиц, сколько пончиков второго типа, нуль, единицы, соответствующие пончикам третьего типа, нуль и единицы для пончиков четвертого типа.

0001111111 0010111111 0011011111 ... 1101101011 ... 1111111000

Суммарное количество единиц в последовательности 7, количество нулей $m = k - 1 = 3$. Длина последовательности $N + m = 7 + 3 = 10$.

Количество таких последовательностей определяется количеством способов записи трех нулей на 10 местах, или $C_{10}^3 = C_{10}^7 = 120$.



$$\begin{matrix} N \\ k \end{matrix} \boxed{C_{N+k-1}^{k-1}}$$

Пример сочетаний с повторениями для $k=3, N=2$:

$\{1, 1\}, \{1, 2\}, \{1, 3\}, \{2, 2\}, \{2, 3\}, \{3, 3\}$

Их количество $\overline{C}_2^3 = C_{2+3-1}^2 = C_{2+3-1}^2 = 6$

Сочетания с повторениями



1. Четверо ребят собрали в лесу 30 белых грибов. Сколькими способами они могут разделить их между собой? 30 грибов, 4 мальчика, 3 разделителя

$$C_{30+4-1}^{4-1} = C_{30+4-1}^{30} \frac{33 * 32 * 31}{1 * 2 * 3} = 5456$$

2. Сколькими способами можно выбрать три рыбины из трех одинаковых щук и трех одинаковых окуней?

Три рыбы, два вида рыб, один разделитель. 0111 1011 1101 1110

Про 3 щуки и 3 окуня сказано, чтобы было понятно, что их достаточно.

$$C_{3+2-1}^{2-1} = C_{3+2-1}^3 = \frac{4 * 3 * 2}{1 * 2 * 3} = 4$$

3. Сколькими способами можно выбрать 13 из 52 игровых карт, различая их только по масти?

13 карт, 4 масти, 3 разделителя

$$C_{13+4-1}^{4-1} = C_{13+4-1}^{13} = C_{16}^3 = \frac{16 * 15 * 14}{1 * 2 * 3} = 560$$



Теорема о числе сочетаний с повторениями

$$\overline{C_n^k} = C_{n+k-1}^k$$

Доказательство

Мы хотим установить взаимно однозначное соответствие между k -сочетаниями с повторениями из n объектов и k -сочетаниями без повторений из $n + k - 1$ объектов.

На примере обсудим способ доказательства этой теоремы: пусть у нас есть число $n = 33$, русский алфавит $\{ а, б, в, ..., ю, я \}$, число $k = 4$. Возьмем 4-сочетание с повторением из алфавита, например $\{ ж, а, б, а \} = \{ а, а, б, ж \}$, образуем для него «паспорт» вида $[1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, ..., 0]$, длина которого равна $36 = 33 + 4 - 1$, перебирая все буквы алфавита, выписывая нули и единицы, где 1 означает, что буква есть, 0 обозначает перегородку между буквами, если буквы нет - то ничего не ставим. Отметим, что между «паспортами» и словами с повторениями есть взаимнооднозначное соответствие, ведь по «паспорту» мы можем восстановить слово.

Теорема о числе сочетаний с повторениями

$$\overline{C_n^k} = C_{n+k-1}^k$$

Доказательство

В общем случае, используя множество $\{a_1, \dots, a_n\}$ создаем k -сочетание с повторениями и затем, создаем для него “паспорт” вида

$$\underbrace{1, \dots, 1, 0, 1, \dots, 1, 0, \dots, 0, 1, \dots, 1}_{n+k-1 \text{ цифр}}$$

количество a_1
количество a_2
количество a_n

так же как мы это делали в примере.

Поскольку мы установили взаимно однозначное соответствие между k -сочетаниями с повторениями из n объектов и k -сочетаниями без повторений из $n + k - 1$ объектов (т.е. “паспортами”), получаем, что интересующее нас равно количеству всех таких “паспортов”.

Каким числом способов мы можем создать “паспорт”, в котором k единиц и $n - 1$ нулей? Мы можем из $n + k - 1$ позиций в векторе выбираем k позиций, на которые поставим 1, на оставшиеся места мы поставим 0. Сделать это можно числом способов.

Сочетания

$$C_n^k = \frac{n!}{k! (n-k)!}$$

Пример

Сколькими способами можно выбрать трёх дежурных из класса, в котором 20 человек?

Решение

$$C_{20}^3 = 1140$$

Ответ: 1140 способами.

Перестановки без повторений

Перестановкой конечного множества S называется упорядоченная последовательность всех его элементов, в которых каждый элемент встречается ровно один раз.

Пример: перестановки множества $S = \{a, b, c\}$

abc, acb, bac, bca, cab, cba

Всего $3! = 6$ перестановок

n

$$n \cdot (n-1) \cdot (n-2) \dots 1 = n!$$

Количество перестановок без повторений

Количество перестановок множества из n элементов

$$P = n!$$

т.к. первый элемент можно выбрать n способами, второй $(n-1)$ способами и т.д.

$$P = n (n-1) (n-2) \dots 2 \cdot 1$$

Перестановки без повторений

Пример

В колледже для концерта подготовили 5 танцевальных композиций. В концертной программе один раз нужно представить каждую композицию. Сколько можно составить концертных программ, если порядок важен?

Решение

Так как количество элементов во множестве неизменно и порядок элементов важен, можно сделать вывод, что нужно вычислить число перестановок: $P_n = n!$, $P_5 = 5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$

Ответ: можно составить 120 различных концертных программ.

Перестановки с повторениями

Если в перестановках участвуют одинаковые предметы, перестановок становится меньше, т.к. часть из них совпадают.

Имеются предметы k различных типов. Сколько перестановок можно сделать из

- N_1 предметов первого типа,
 - N_2 – предметов второго типа,
 - ...,
 - N_k – предметов k -го типа
- Всего ($N_1 + N_2 + \dots + N_k = N$) предметов

3

k

3 | 3 | 3

k | k | k | k

k

n

$\begin{matrix} 0_1 \\ 0_k \end{matrix}$

$\begin{matrix} 0_k \\ 0_1 \end{matrix}$

k^n

Примеры

Сколько различных буквосочетаний можно получить из букв слова *отец*?

Ответ: 4!

Из букв слова *папа*? Ответ: $4! / X = 4! / 2! / 2!$

$$\frac{N!}{N_1! N_2! \dots}$$

Перестановки с повторениями

Примеры

1. Сколько различных буквосочетаний можно получить из букв слова *капкан*?

2. Сколькими способами можно расположить в ряд 2 одинаковых яблока и 3 одинаковые груши?

3. Сколько различных сообщений можно закодировать, меняя порядок шести флажков: 2 красных, 3 синих и 1 зеленого?

4. Для выполнения работы необходимо в некотором порядке выполнить дважды каждое из трех действий (*a*, *b*, *c*). Например: *aabbcc*, *cabbac*. Сколько существует различных способов выполнения работы?

$$\frac{5!}{2! \cdot 3!} = 10$$
$$\frac{6!}{2! \cdot 3! \cdot 1!} = 210$$

Количество перестановок с повторениями

$$P(N_1, N_2, \dots, N_k) = \frac{N!}{N_1! N_2! \dots N_k!}, N = N_1 + N_2 + \dots + N_k$$

$$P(N_1, N_2, \dots, N_k) = C_n^{N_1} C_{N-N_1}^{N_2} \dots C_{N-N_1-\dots-N_{k-1}}^{N_k}$$

Разбиения

Разбиение числа n — это представление n в виде суммы положительных целых чисел, называемых частями. Порядок следования частей не учитывается (в отличие от композиций), то есть разбиения, отличающиеся только порядком частей, считаются равными. В записи разбиения части обычно перечисляются в невозрастающем порядке. При записи разбиений обязательно надо указывать, в каком порядке их записывать.

Разбиения числа 5 записаны

- в лексикографическом порядке,
- слагаемые записаны в невозрастающем порядке

Пример: Разбиения числа 5

(1,1,1,1,1),

(2,1,1,1),

(2,2,1),

(3,1,1),

(3,2),

(4,1),

(5)

Количество разбиений

Подсчитать число разбиений конечного множества предметов S , где $|S|=N$ (количество предметов), на k различных попарно не пересекающихся подмножеств $S = S_1 \cup S_2 \cup \dots \cup S_k$

$$P(n, k) = P(n - 1, k - 1) + P(n - k, k)$$

$$P(i, i) = 1, \forall i \in N$$

$$P(i, 1) = 1 \quad \forall i \in N$$

$$P(i, j) = 0, \forall j > i$$

Композиции

- Если порядок чисел p_i важен, то $(p_1, p_2, \dots, p_k)$ называется **композицией n** . Поиск композиций ведется с ограничением $p_i > 0$.
- Если k фиксировано, то такие композиции называются **композициями N из k частей**.
- При их поиске ограничение $p_i > 0$ может сниматься, т.е. разрешается $p_i \geq 0$.
- Если порядок p_i не важен и $p_i > 0$, то $\{p_1, p_2, \dots, p_k\}$ является мультимножеством и называется **разбиением N** .

Количество композиций

Число композиций N равно 2^{N-1}

Число композиций N из k частей с ограничением $p_i > 0$ равно C_{N-1}^{k-1}

Число композиций N из k частей, если $p_i \geq 0$ равно C_{N+k-1}^{k-1}

При $N=4$, $k=2$ число композиций 5

При $N=4$, $k=3$ число композиций 15 (можно использовать 1 или 2 нуля на разных местах)

$$4 = 4 + 0$$

$$4 = 3 + 1$$

$$4 = 2 + 2$$

$$4 = 1 + 3$$

$$4 = 0 + 4$$

$$4 = 3 + 1$$

$$4 = 2 + 2$$

$$4 = 1 + 3$$

$$4 = 4$$

$$4 = 3 + 1$$

$$4 = 2 + 2$$

$$4 = 2 + 1 + 1$$

$$4 = 1 + 3$$

$$4 = 1 + 2 + 1$$

$$4 = 1 + 1 + 2$$

$$4 = 1 + 1 + 1 + 1$$

Композиции и разбиения

Композиции и разбиения. Пусть стоит задача порождения разбиения положительного числа N в последовательность неотрицательных целых чисел $\{p_1, p_2, \dots, p_k\}$, так что

$$N = p_1 + p_2 + \dots + p_k$$

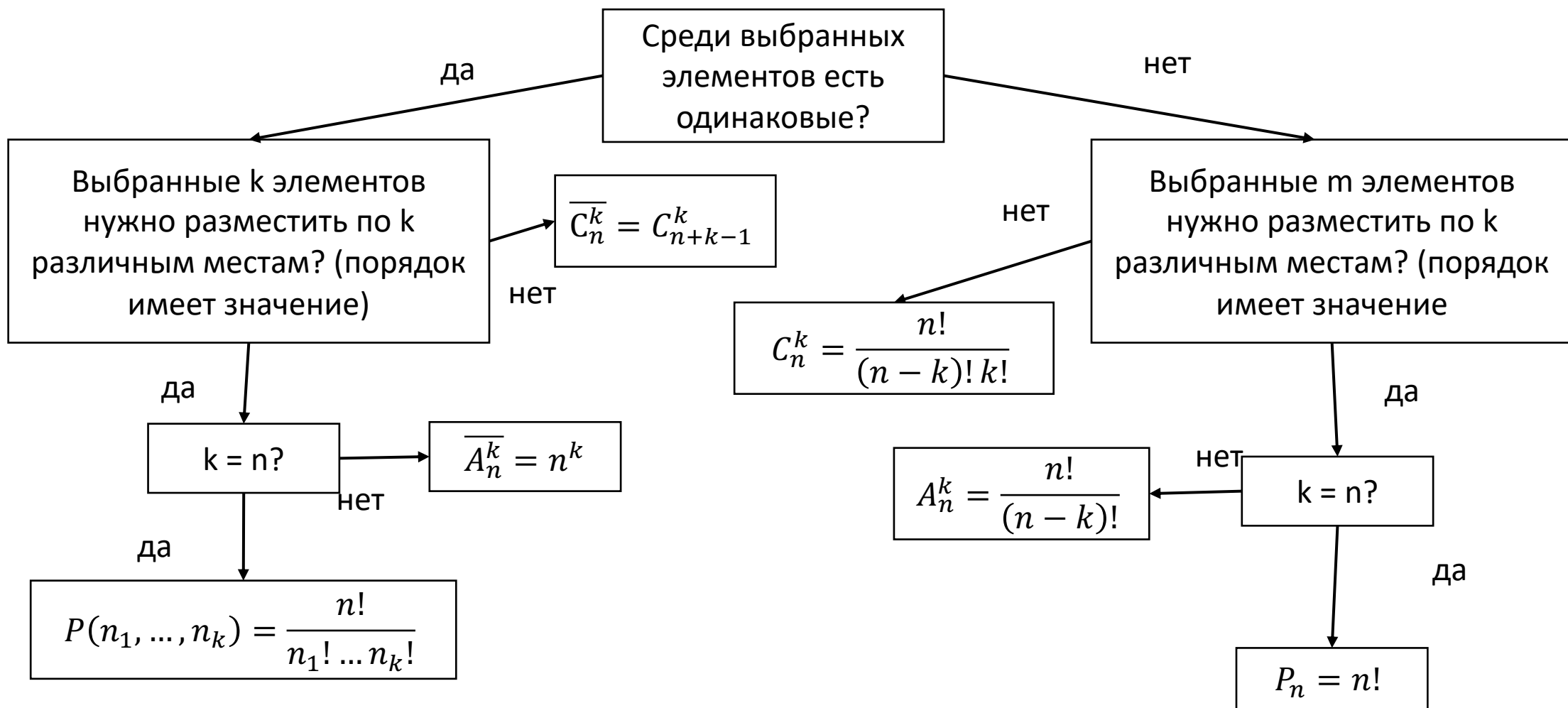
причем на p_i могут накладываться различные ограничения.

Поясним различие между композициями, композициями из k частей и разбиениями на примере $N=3$,

композиции:	(3), (1,2), (2,1), (1,1,1)
композиции из двух частей $k=2$, ограничение $p_i > 0$:	(1,2), (2,1)
композиции из двух частей $k=2$, ограничение $p_i \geq 0$:	(0,3), (1,2), (2,1), (3,0)
разбиения:	{3}, {1,2}, {1,1,1}

Количество комбинаторных объектов

Перестановки из N элементов	$N!$
Перестановки из N элементов с повторениями	$P(N_1, N_2, \dots, N_k) = \frac{N!}{N_1! N_2! \dots N_k!}$
Размещения из N элементов по M	$A_N^M = \frac{N!}{(N-M)!}$
Размещения длины k из элементов N типов с повторениями	$\overline{A_N^k} = N^k$
Сочетания из N элементов по M	$C_N^M = \binom{N}{M} = \frac{N!}{M! (N-M)!}$
Сочетания N элементов из элементов k типов с повторениями	$\overline{C_N^k} = C_{N+k-1}^N = C_{N+k-1}^{k-1}$
Композиции N	2^{N-1}
Композиции N из k частей при $p_i > 0$	C_{N-1}^{k-1}
Композиции N из k частей при $p_i \geq 0$	C_{N+k-1}^{k-1}
Разбиение N	Ассимпт.ф-лы



Задачи генерации комбинаторных объектов

Условие

Если имеется N объектов, то задается взаимно однозначное соответствие между целыми числами $0, 1, \dots, N - 1$ и N объектами. Работаем с целыми числами от 0 до $N - 1$, которыми пронумерованы объекты.

Задачи

1. Присвоить конфигурации индекс или получить конфигурацию по индексу
2. Определить общее количество возможных конфигураций (перестановок || сочетаний || разбиений || композиций и т. д.)
3. Сгенерировать все конфигурации
4. Найти следующую || предыдущую конфигурацию
5. Сгенерировать случайную конфигурацию

Подходы к порождению комбинаторных объектов

При порождении комбинаторных объектов существуют два подхода:

1. Систематическое порождение всех возможных конфигураций (или следующей конфигурации)
2. Порождение равномерно распределенных случайных конфигураций. (Случайные конфигурации используются, например, при порождении перестановок. Нужен хороший генератор случайных чисел, дающий равномерное распределение. Здесь не рассматриваем.)

Генерация перестановок

- **Вход** – целое число n
- **Задачи**: сгенерировать
 1. все возможные перестановки
 2. очередную перестановку
 3. случайную перестановку
- Основное – понятие порядка, т.е. последовательности, в которой перестановки создаются.
 1. Пример: лексикографический порядок.
 2. Еще подход: один обмен элементов для получения следующей перестановки.
 3. Алгоритм Джонсона-Троттера – один обмен соседних элементов.

Лексикографический порядок

Лексикографический порядок на множестве перестановок:

$P^{(1)} < P^{(2)}$ тогда и только тогда, когда существует такое $t \geq 1$, что $P^{(1)}[t] \leq P^{(2)}[t]$ и $P^{(1)}[i] = P^{(2)}[i]$ для всех $i < t$

Пример. $t=5, i=1, 2, 3, 4$

2345**6**123

2345**7**213

Совпадает начало массива

Систематическое порождение комбинаторных объектов

Схема алгоритма систематического порождения :

```
выбрать начальную конфигурацию;  
while not условие окончания do  
    вывести объект;  
    преобразовать текущий объект в следующий;
```

Замечание. При такой схеме алгоритм выполняет еще одно преобразование после того, как выведен последний объект.

преобразовать текущий объект в следующий:

- 1. Алгоритмы минимального изменения:** изменения текущего объекта при переходе к следующему должны быть по возможности малыми, например, обмен местами 2 элементов
- 2. Перечисление целых чисел в каком-либо порядке** (лексикографическом, антилексикографическом и т.д.)

Алгоритм порождения перестановок в лексикографическом порядке

- Начинаем с перестановки $P=(1, 2, \dots, N)$,
- Переходим к следующей путем просмотра текущей справа налево с целью поиска самой правой позиции, в которой $p_i < p_{i+1}$.
- Ищем p_j , наименьший элемент, расположенный справа от p_i и больше его.
- Затем осуществляется транспозиция элементов p_i и p_j
- отрезок $p_{i+1}, \dots, p_N$ (элементы которого расположены в порядке убывания) «переворачивается».
- Последняя перестановка $P=(N, N-1, \dots, 2, 1)$

```
Init;  
//Инициализация: в массиве P - первая  
перестановка, в массиве Last - последняя  
print; //Вывод перестановки  
while not(Eq(P, Last)) do  
    begin  
        //Функция Eq определяет равенство текущей  
перестановки и последней; если не равны, то  
генерируется следующая  
        GetNext; //получение следующей  
перестановки  
        print;  
    end;
```

Получение следующей перестановки в лексикографическом порядке

```
bool NextSet(int *a, int n)
{
    int j = n - 2;
    while (j != -1 && a[j] >= a[j + 1]) j--;
    if (j == -1)
        return false; // больше перестановок нет
    int k = n - 1;
    while (a[j] >= a[k]) k--;
    swap(a, j, k);
    int l = j + 1, r = n - 1; // сортируем
    оставшуюся часть последовательности
    while (l < r)
        swap(a, l++, r--);
    return true;
}

void Print(int *a, int n) // вывод перестановки
{
    static int num = 1; // номер перестановки
    cout.width(3); // ширина поля вывода номера
    перестановки
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, *a;
    cout << "N = ";
    cin >> n;
    a = new int[n];
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
    Print(a, n);
    while (NextSet(a, n))
        Print(a, n);
    return 0;
}
```

Результат выполнения

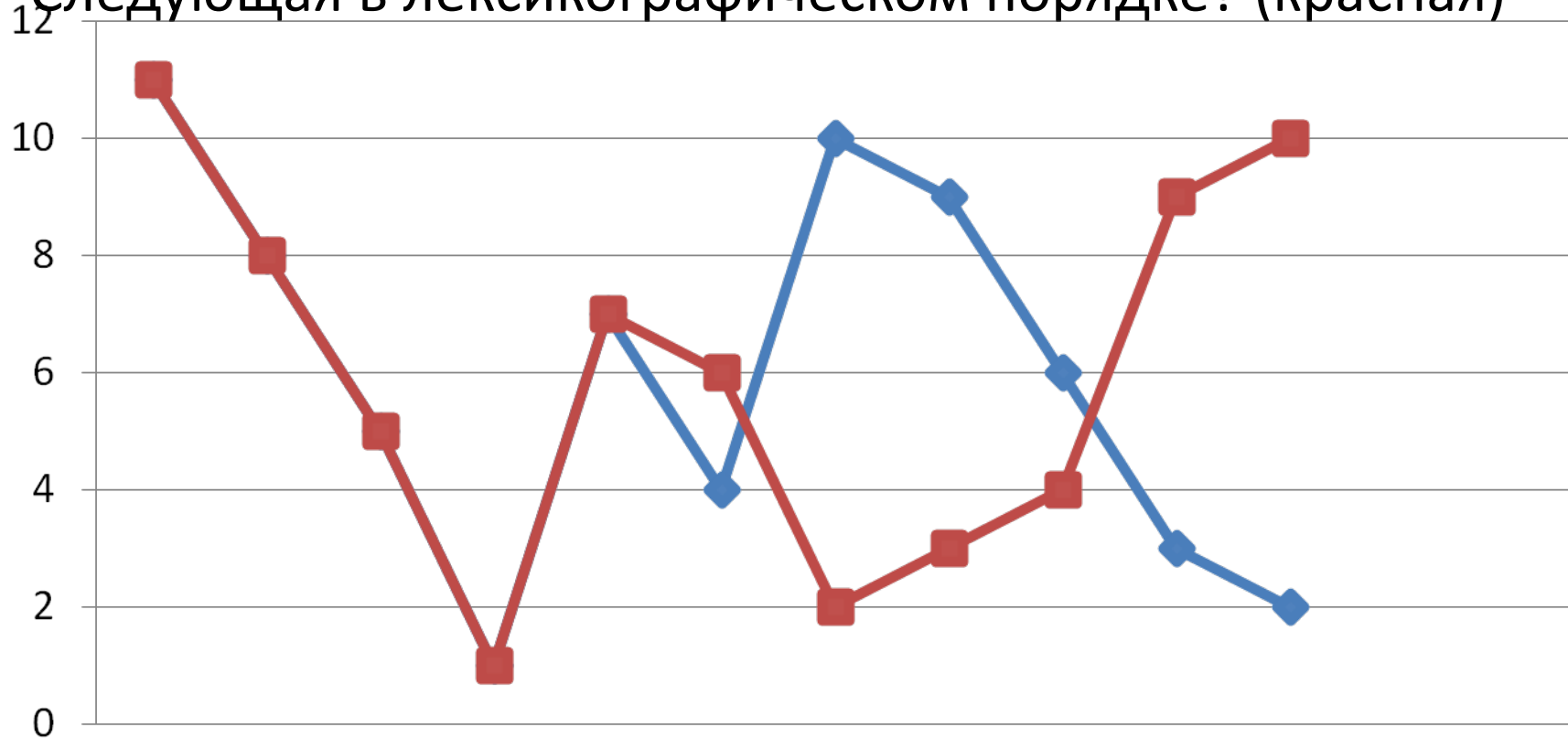
C:\Projects\MyProgram\Debug\MyProgram.exe

N = 4

```
1: 1 2 3 4
2: 1 2 4 3
3: 1 3 2 4
4: 1 3 4 2
5: 1 4 2 3
6: 1 4 3 2
7: 2 1 3 4
8: 2 1 4 3
9: 2 3 1 4
10: 2 3 4 1
11: 2 4 1 3
12: 2 4 3 1
13: 3 1 2 4
14: 3 1 4 2
15: 3 2 1 4
16: 3 2 4 1
17: 3 4 1 2
18: 3 4 2 1
19: 4 1 2 3
20: 4 1 3 2
21: 4 2 1 3
22: 4 2 3 1
23: 4 3 1 2
24: 4 3 2 1
```


Алгоритм генерации перестановки

- Исходная перестановка 11, 8, 5, 1, 7, 4, 10, 9, 6, 3, 2 (синяя)
- Следующая в лексикографическом порядке? (красная)



Получение следующей перестановки с повторениями в лексикографическом порядке

```
bool NextSet(int *a, int n)
{
    int j = n - 2;
    while (j != -1 && a[j] >= a[j + 1]) j--;
    if (j == -1)
        return false; // больше перестановок нет
    int k = n - 1;
    while (a[j] >= a[k]) k--;
    swap(a, j, k);
    int l = j + 1, r = n - 1; // сортируем
    оставшуюся часть последовательности
    while (l < r)
        swap(a, l++, r--);
    return true;
}

void Print(int *a, int n) // вывод перестановки
{
    static int num = 1; // номер перестановки
    cout.width(3); // ширина поля вывода номера
    перестановки
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, *a;
    cout << "N = ";
    cin >> n;
    a = new int[n];
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
    a[1] = 1; // повторяющийся элемент
    Print(a, n);
    while (NextSet(a, n))
        Print(a, n);
    cin.get(); cin.get();
    return 0;
}
```

Результат работы приведенного выше алгоритма:

C:\Projects\MyProgram\Debug\MyProgram.exe

N = 4

```
1: 1 1 3 4
2: 1 1 4 3
3: 1 3 1 4
4: 1 3 4 1
5: 1 4 1 3
6: 1 4 3 1
7: 3 1 1 4
8: 3 1 4 1
9: 3 4 1 1
10: 4 1 1 3
11: 4 1 3 1
12: 4 3 1 1
```

Перестановки с одним обменом

- heapPermute(N)
- **if** N == 0 **then**
- **ВЫВОД** (массив P)
- **else**
- **for** i ← 1 **to** N **do**
- heapPermute(N - 1)
- **if** N mod 2 == 1 **then** swap(P[1], P[N])
- **else** swap(P[i], P[N])
- **end if**
- **end for**
- **end if**

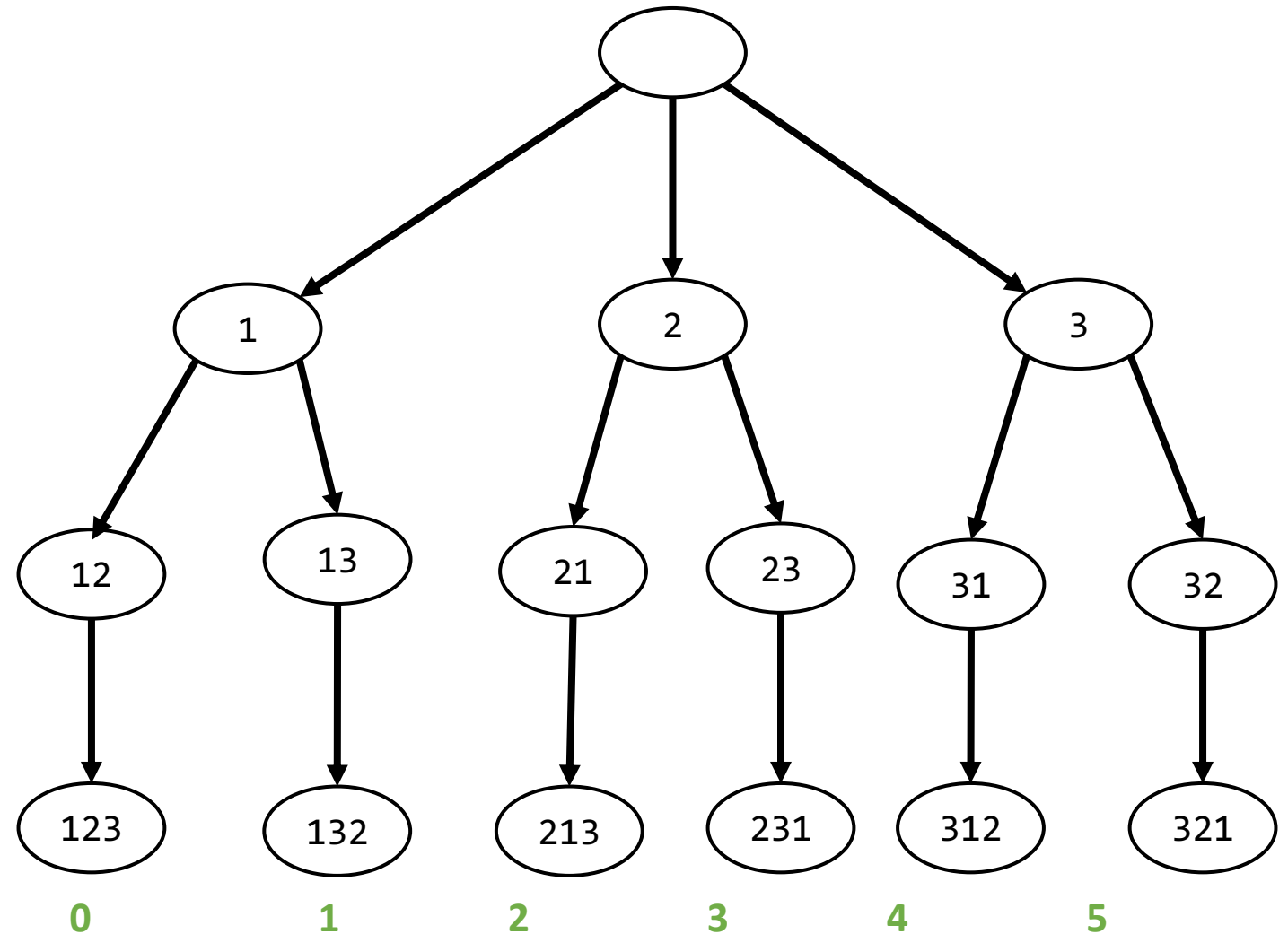
Допустим, построена последовательность перестановок из $(n-1)$ элементов.

Расширим каждую из $(n-1)!$ этих перестановок путем вставки элемента n на каждое из n возможных мест.

(Причем элемент n добавляется к P_i последовательно во все позиции справа налево, если i нечетное, и слева направо, если i четное)

Представление в виде графа

- Генерация комбинаторных объектов основана на переборе всех возможных комбинаций символов алфавита по определенным правилам. Процесс этого перебора можно представить в виде графа.
- Например, перестановки длины $n = 3$.



Генерация перестановки по номеру

```
function permutation_to_number(p, number) {  
    if (length(p) == n) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (!used(c)) {  
            t = factorial(n - length(p) - 1);  
            if (number < t) {  
                permutation_to_number(p + c, number);  
                return;  
            }  
            else {  
                number -= t;  
            }  
        }  
    }  
}
```

- Для хранения уже использованных символов алфавита можно создать массив типа boolean.
- Подсчет факториалов от 0 до n можно произвести заранее или использовать динамическое программирование.

Генерация перестановки по номеру

permutation_from_number([], 10)

n = 4
number = 10
p = []



used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_to_number(p, number) {  
  if (length(p) == n) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (!used(c)) {  
      t = factorial(n - length(p) - 1);  
      if (number < t) {  
        permutation_to_number(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```

Генерация перестановки по номеру

n = 4
number = 10
p = []

пропускаем эту ветку, так как
в ней не находится искомая
перестановка

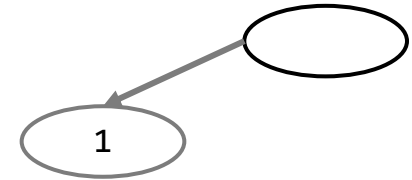
permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

number = $10 - 6 = 4$



used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_to_number(p, number) {  
  if (length(p) == n) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (!used(c)) {  
      t = factorial(n - length(p) - 1);  
      if (number < t) {  
        permutation_to_number(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```


Генерация перестановки по номеру

n = 4
number = 4
p = []

продолжаем идти по этой ветке

used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_from_number(p, number) {  
  if (length(p) == n) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (!used(c)) {  
      t = factorial(n - length(p) - 1);  
      if (number < t) {  
        permutation_from_number(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

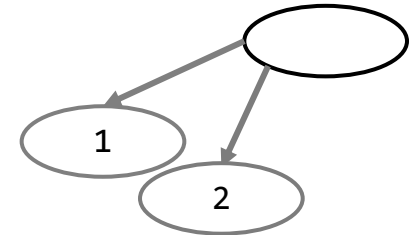
$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4)



Генерация перестановки по номеру

n = 4
number = 4
p = [2]

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_from_number(p, number) {  
  if (length(p) == n) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (!used(c)) {  
      t = factorial(n - length(p) - 1);  
      if (number < t) {  
        permutation_from_number(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

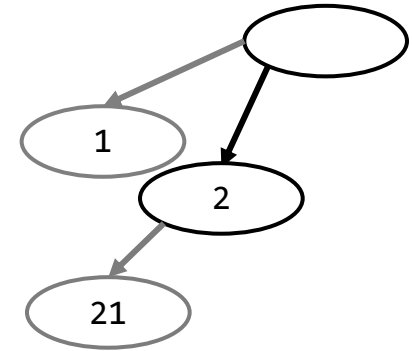
permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

$number = 4 - 2 = 2$



Генерация перестановки по номеру

n = 4
number = 2
p = [2]

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_from_number(p, number) {
  if (length(p) == n) {
    print(p);
    return;
  }
  for (c in A) {
    if (!used(c)) {
      t = factorial(n - length(p) - 1);
      if (number < t) {
        permutation_from_number(p + c, number);
        return;
      }
      else {
        number -= t;
      }
    }
  }
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

number = $10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

number = $4 - 2 = 2$

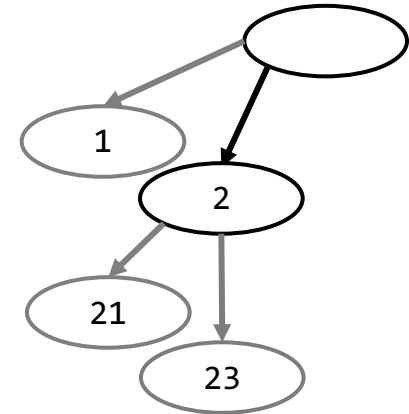
c = 2 is used

c = 3 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$2 \geq 2:$

number = $2 - 2 = 0$



Генерация перестановки по номеру

n = 4
number = 0
p = [2]

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_from_number(p, number) {  
  if (length(p) == n) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (!used(c)) {  
      t = factorial(n - length(p) - 1);  
      if (number < t) {  
        permutation_from_number(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

$number = 4 - 2 = 2$

c = 2 is used

c = 3 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$2 \geq 2:$

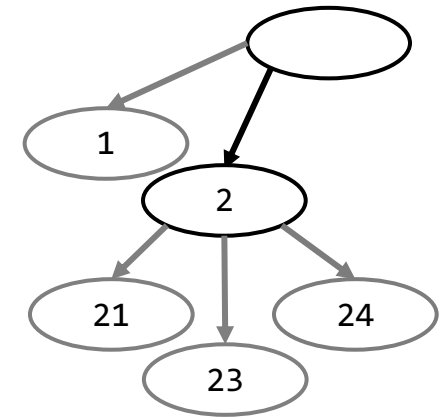
$number = 2 - 2 = 0$

c = 4 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$0 < 2:$

permutation_from_number([2,4], 0)



Генерация перестановки по номеру

n = 4
number = 0
p = [2, 4]

used(1)	used(2)	used(3)	used(4)
false	true	false	true

```
function permutation_from_number(p, number) {
  if (length(p) == n) {
    print(p);
    return;
  }
  for (c in A) {
    if (!used(c)) {
      t = factorial(n - length(p) - 1);
      if (number < t) {
        permutation_from_number(p + c, number);
        return;
      }
      else {
        number -= t;
      }
    }
  }
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

$number = 4 - 2 = 2$

c = 2 is used

c = 3 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$2 \geq 2:$

$number = 2 - 2 = 0$

c = 4 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$0 < 2:$

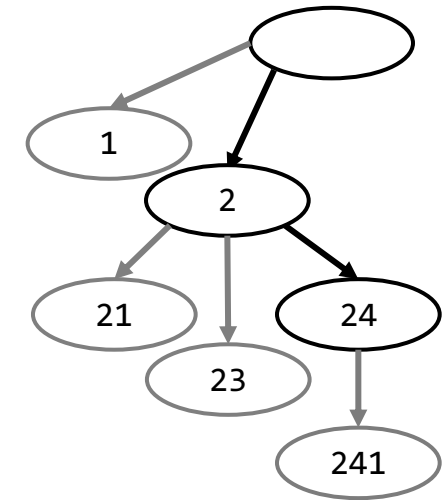
permutation_from_number([2,4], 0):

c = 1 is not used:

$t = (4 - 3)! = 1! = 1$

$0 < 1:$

permutation_from_number([2,4,1], 0)



Генерация перестановки по номеру

n = 4
number = 0
p = [2, 4, 1]

used(1)	used(2)	used(3)	used(4)
true	true	false	true

```
function permutation_from_number(p, number) {
  if (length(p) == n) {
    print(p);
    return;
  }
  for (c in A) {
    if (!used(c)) {
      t = factorial(n - length(p) - 1);
      if (number < t) {
        permutation_from_number(p + c, number);
        return;
      }
      else {
        number -= t;
      }
    }
  }
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

$number = 4 - 2 = 2$

c = 2 is used

c = 3 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$2 \geq 2:$

$number = 2 - 2 = 0$

c = 4 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$0 < 2:$

permutation_from_number([2,4], 0):

c = 1 is not used:

$t = (4 - 3)! = 1! = 1$

$0 < 1:$

permutation_from_number([2,4,1], 0):

c = 1 is used

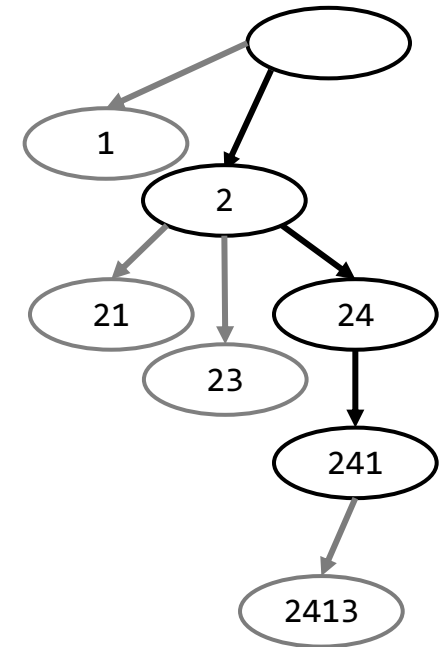
c = 2 is used

c = 3 is not used:

$t = (4 - 4)! = 0! = 1$

$0 < 1:$

permutation_from_number([2,4,1,3], 0)



Генерация перестановки по номеру

n = 4
number = 0
p = [2, 4, 1, 3]

длина очередного префикса
равна n = 4, выводим ответ и
выходим из рекурсии

used(1)	used(2)	used(3)	used(4)
true	true	true	true

```
function permutation_from_number(p, number) {
  if (length(p) == n) {
    print(p);
    return;
  }
  for (c in A) {
    if (!used(c)) {
      t = factorial(n - length(p) - 1);
      if (number < t) {
        permutation_from_number(p + c, number);
        return;
      }
      else {
        number -= t;
      }
    }
  }
}
```

permutation_from_number([], 10):

c = 1 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$10 \geq 6:$

$number = 10 - 6 = 4$

c = 2 is not used:

$t = (4 - 0 - 1)! = 3! = 6$

$4 < 6:$

permutation_from_number([2], 4):

c = 1 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$4 \geq 2:$

$number = 4 - 2 = 2$

c = 2 is used

c = 3 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$2 \geq 2:$

$number = 2 - 2 = 0$

c = 4 is not used:

$t = (4 - 1 - 1)! = 2! = 2$

$0 < 2:$

permutation_from_number([2,4], 0):

c = 1 is not used:

$t = (4 - 3)! = 1! = 1$

$0 < 1:$

permutation_from_number([2,4,1], 0):

c = 1 is used

c = 2 is used

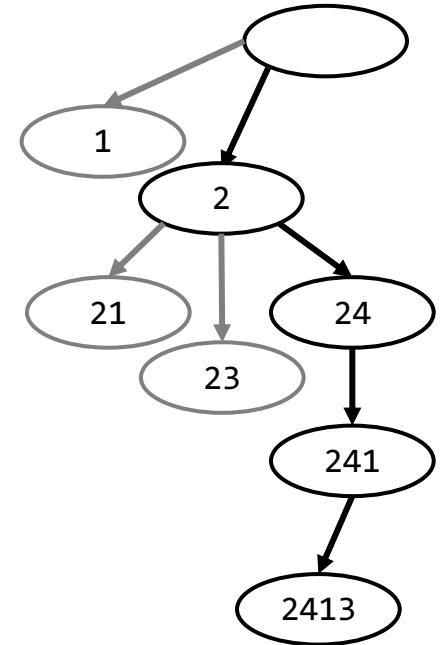
c = 3 is not used:

$t = (4 - 4)! = 0! = 1$

$0 < 1:$

permutation_from_number([2,4,1,3], 0):

length([2,4,1,3]) = 4



Получение номера перестановки

```
function permutation_to_number(p, number, o) {  
  if (length(p) == n) {  
    print(number);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      if (is_prefix(p + c, o)) {  
        permutation_to_number(p + c, number, o);  
        return;  
      }  
      else {  
        number += factorial(n - length(p) - 1);  
      }  
    }  
  }  
}
```

- Для хранения уже использованных символов алфавита можно создать массив типа boolean.
- Подсчет факториалов от 0 до n можно произвести заранее или использовать динамическое программирование.
- Один из вариантов функции `is_prefix(prefix, object)` {
 index = length(prefix)
 return prefix[index] == object[index]
}

Получение номера перестановки

permutation_to_number(p, number, o)



n = 4
o = [2,4,1,3]

p = []
number = 0

used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

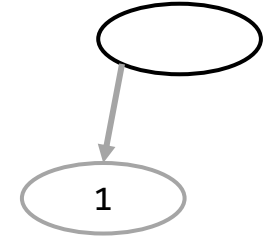
Получение номера перестановки

$n = 4$ $p = []$
 $o = [2, 4, 1, 3]$ $number = 0$

permutation_to_number(p, number, o):

$c = 1$ is not used:

$p = [1]$ is not prefix:
 $number = (4 - 1)! = 6$



used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

Получение номера перестановки

$n = 4$ $p = []$
 $o = [2, 4, 1, 3]$ $number = 6$

used(1)	used(2)	used(3)	used(4)
false	false	false	false

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

permutation_to_number(p, number, o):

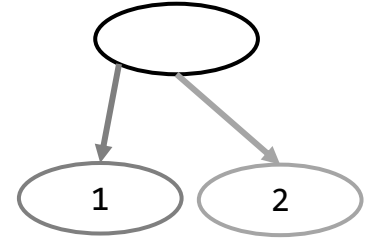
c = 1 is not used:

$p = [1]$ is not prefix:
 $number = (4 - 1)! = 6$

c = 2 is not used:

$p = [2]$ is prefix:

permutation_to_number([2], number, o)



Получение номера перестановки

$n = 4$ $p = [2]$
 $o = [2, 4, 1, 3]$ $number = 6$

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

$p = [1]$ is not prefix:
 $number = (4 - 1)! = 6$

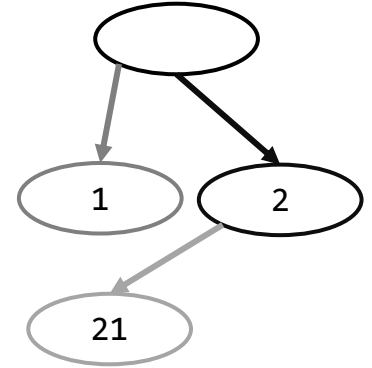
c = 2 is not used:

$p = [2]$ is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

$p = [2, 1]$ is not prefix:
 $number = 6 + (4 - 2)! = 8$



Получение номера перестановки

$n = 4$

$p = [2]$

$o = [2, 4, 1, 3]$

number = 8

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_to_number(p, number, o) {  
  if (length(p) == n) {  
    print(number);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      if (is_prefix(p + c, o)) {  
        permutation_to_number(p + c, number, o);  
        return;  
      }  
      else {  
        number += factorial(n - length(p) - 1);  
      }  
    }  
  }  
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

p = [1] is not prefix:

number = $(4 - 1)! = 6$

c = 2 is not used:

p = [2] is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

p = [2, 1] is not prefix:

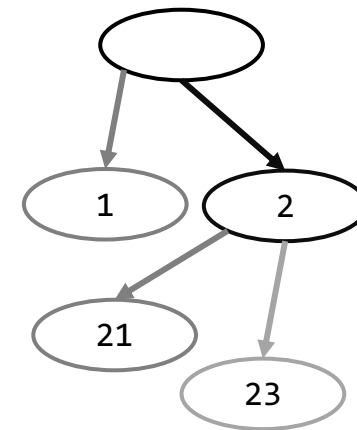
number = $6 + (4 - 2)! = 8$

c = 2 is used

c = 3 is not used:

p = [2, 3] is not prefix:

number = $8 + (4 - 2)! = 10$



Получение номера перестановки

$n = 4$ $p = [2]$
 $o = [2, 4, 1, 3]$ $number = 10$

used(1)	used(2)	used(3)	used(4)
false	true	false	false

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

p = [1] is not prefix:
number = $(4 - 1)! = 6$

c = 2 is not used:

p = [2] is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

p = [2, 1] is not prefix:
number = $6 + (4 - 2)! = 8$

c = 2 is used

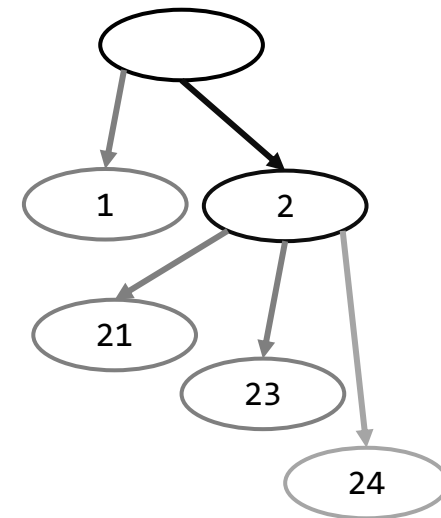
c = 3 is not used:

p = [2, 3] is not prefix:
number = $8 + (4 - 2)! = 10$

c = 4 is not used:

p = [2, 4] is prefix:

permutation_to_number([2, 4], 10, o)



Получение номера перестановки

$n = 4$ $p = [2, 4]$
 $o = [2, 4, 1, 3]$ $number = 10$

used(1)	used(2)	used(3)	used(4)
false	true	false	true

```
function permutation_to_number(p, number, o) {  
    if (length(p) == n) {  
        print(number);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix(p + c, o)) {  
                permutation_to_number(p + c, number, o);  
                return;  
            }  
            else {  
                number += factorial(n - length(p) - 1);  
            }  
        }  
    }  
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

p = [1] is not prefix:
number = $(4 - 1)! = 6$

c = 2 is not used:

p = [2] is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

p = [2, 1] is not prefix:
number = $6 + (4 - 2)! = 8$

c = 2 is used

c = 3 is not used:

p = [2, 3] is not prefix:
number = $8 + (4 - 2)! = 10$

c = 4 is not used:

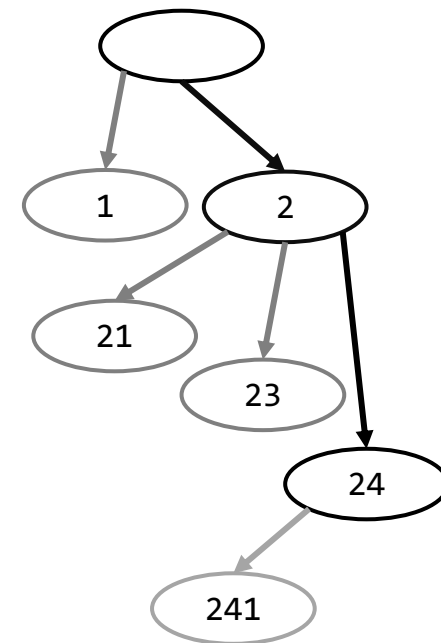
p = [2, 4] is prefix:

permutation_to_number([2, 4], number, o):

c = 1 is not used:

p = [2, 4, 1] is prefix:

permutation_to_number([2, 4, 1], number, o)



Получение номера перестановки

$n = 4$

$p = [2, 4, 1]$

$o = [2, 4, 1, 3]$

number = 10

used(1)	used(2)	used(3)	used(4)
true	true	false	true

```
function permutation_to_number(p, number, o) {  
  if (length(p) == n) {  
    print(number);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      if (is_prefix(p + c, o)) {  
        permutation_to_number(p + c, number, o);  
        return;  
      }  
      else {  
        number += factorial(n - length(p) - 1);  
      }  
    }  
  }  
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

p = [1] is not prefix:
number = $(4 - 1)! = 6$

c = 2 is not used:

p = [2] is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

p = [2, 1] is not prefix:
number = $6 + (4 - 2)! = 8$

c = 2 is used

c = 3 is not used:

p = [2, 3] is not prefix:
number = $8 + (4 - 2)! = 10$

c = 4 is not used:

p = [2, 4] is prefix:

permutation_to_number([2, 4], number, o):

c = 1 is not used:

p = [2, 4, 1] is prefix:

permutation_to_number([2, 4, 1], number, o):

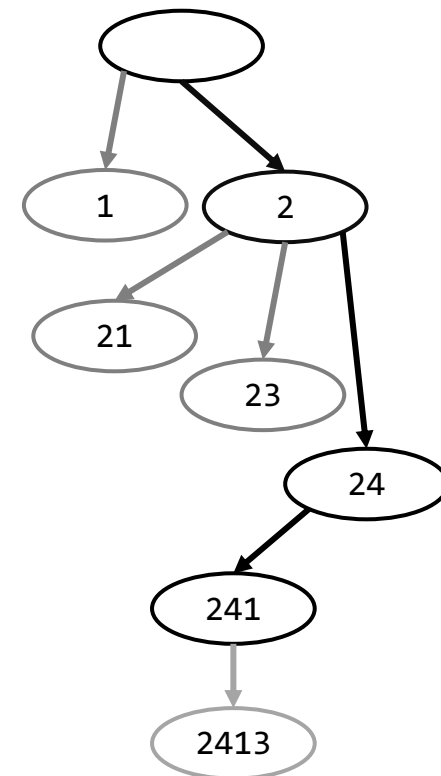
c = 1 is used

c = 2 is used

c = 3 is not used:

p = [2, 4, 1, 3] is prefix:

permutation_to_number([2, 4, 1, 3], number, o)



Получение номера перестановки

$n = 4$

$p = [2, 4, 1, 3]$

$o = [2, 4, 1, 3]$

number = 10

used(1)	used(2)	used(3)	used(4)
true	true	true	true

```
function permutation_to_number(p, number, o) {
    if (length(p) == n) {
        print(number);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            if (is_prefix(p + c, o)) {
                permutation_to_number(p + c, number, o);
                return;
            }
            else {
                number += factorial(n - length(p) - 1);
            }
        }
    }
}
```

permutation_to_number(p, number, o):

c = 1 is not used:

p = [1] is not prefix:
 $\text{number} = (4 - 1)! = 6$

c = 2 is not used:

p = [2] is prefix:

permutation_to_number([2], number, o):

c = 1 is not used:

p = [2, 1] is not prefix:
 $\text{number} = 6 + (4 - 2)! = 8$

c = 2 is used

c = 3 is not used:

p = [2, 3] is not prefix:
 $\text{number} = 8 + (4 - 2)! = 10$

c = 4 is not used:

p = [2, 4] is prefix:

permutation_to_number([2, 4], number, o):

c = 1 is not used:

p = [2, 4, 1] is prefix:

permutation_to_number([2, 4, 1], number, o):

c = 1 is used

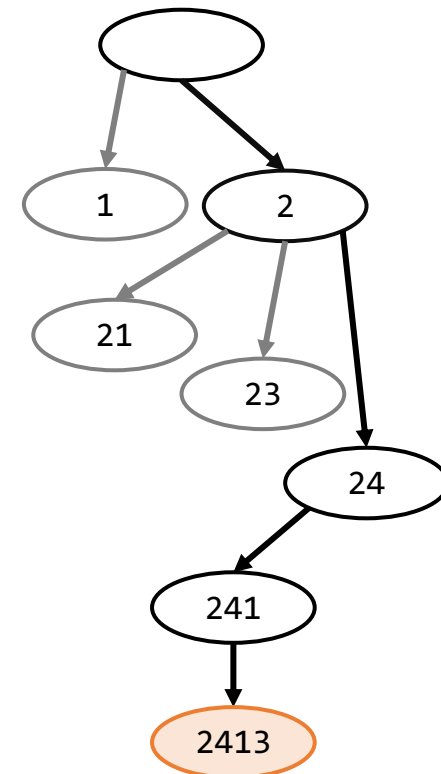
c = 2 is used

c = 3 is not used:

p = [2, 4, 1, 3] is prefix:

permutation_to_number([2, 4, 1, 3], number, o):

length(p) = n = 4



Генерация размещений без повторений в лексикографическом порядке

- Пример. $N=4$, $M=3$
- 123 124 132 134 142 143 213 214 231 234

241 243 312 314 321 324 341 342

412 413 421 423 431 432

Генерация размещений без повторений

```
• Solve (t: integer); //t - номер позиции в размещении
• for i ← 1 to n do //Перебираем цифры и находим не
•   if not (i in S) then
•     S ← S + [i]; //Запоминаем её в множестве занятых
•     A[t] ← i; //Записываем её в размещение
•     if t < m then Solve(t + 1) else Print;
•     // Если размещение не получено, то идём на следующую
позиции,
•     // иначе выводим очередное размещение
•     S ← S - [i]; //Возвращаем цифру в неиспользованные
•   Endif;
• End;
```

A – массив для хранения размещений
[1..M]

S – множество для хранения
использованных в размещении чисел
t – номер позиции в размещении

Основная программа

S = [];

Solve(1);

Генерация размещений без повторений

```
bool NextSet(int *a, int n, int m)
{
    int j;
    do // повторяем пока не будет найдено следующее
       размещение
    {
        j = n - 1;
        while (j != -1 && a[j] >= a[j + 1]) j--;
        if (j == -1)
            return false; // больше размещений нет
        int k = n - 1;
        while (a[j] >= a[k]) k--;
        swap(a, j, k);
        int l = j + 1, r = n - 1; // сортируем оставшуюся
        часть последовательности
        while (l < r)
            swap(a, l++, r--);
    } while (j > m - 1);
    return true;
}

void Print(int *a, int n) // вывод размещения
{
    static int num = 1; // номер размещения
    cout.width(3); // ширина поля вывода номера размещения
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, m, *a;
    cout << "N = ";
    cin >> n;
    cout << "M = ";
    cin >> m;
    a = new int[n];
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
    Print(a, m);
    while (NextSet(a, n, m))
        Print(a, m);
    cin.get(); cin.get();
    return 0;
}
```

Результат выполнения

C:\Projects\MyProgram\Debug\MyProgram.exe

N = 4

M = 3

```
1:  1 2 3
2:  1 2 4
3:  1 3 2
4:  1 3 4
5:  1 4 2
6:  1 4 3
7:  2 1 3
8:  2 1 4
9:  2 3 1
10: 2 3 4
11: 2 4 1
12: 2 4 3
13: 3 1 2
14: 3 1 4
15: 3 2 1
16: 3 2 4
17: 3 4 1
18: 3 4 2
19: 4 1 2
20: 4 1 3
21: 4 2 1
22: 4 2 3
23: 4 3 1
24: 4 3 2
```

Генерация размещений с повторениями в лексикографическом порядке

Пример. $N=4, k=3$

111 112 113 114 121 122 123 124 131 132 133 134 141 142 143
144
???
???
??? 444

Общее количество размещений с повторениями

$$\overline{A_N^k} = N^k$$

Здесь

$$\overline{A_4^3} = 4^3 = 64$$



Генерация размещений с повторениями в лексикографическом порядке

- A – массив для хранения текущего размещения $[1..k]$
- s – номер позиции в размещении
- Первая последовательность $\langle 1\ 1\ \dots\ 1 \rangle$
- Последняя последовательность $\langle N\ N\ \dots\ N \rangle$
- Как перейти от A к следующей последовательности?
- У следующей последовательности первые s элементов должны быть такие же, а $(s+1)$ -й элемент – больше. Это возможно, если в текущей последовательности $a[s+1] < N$.
- Надо выбрать наибольшее такое s и увеличить $a[s+1]$ на единицу.
- Т.е. двигаясь справа налево найти самый правый элемент, меньший N , увеличить его на 1, а все элементы последовательности, идущие за ним, положить равными единице.
- Примеры: $144 \rightarrow 211$, $134 \rightarrow 141$

Генерация размещений с повторениями

```
bool NextSet(int *a, int n, int m)
{
    int j = m - 1;
    while (j >= 0 && a[j] == n) j--;
    if (j < 0) return false;
    if (a[j] >= n)
        j--;
    a[j]++;
    if (j == m - 1) return true;
    for (int k = j + 1; k < m; k++)
        a[k] = 1;
    return true;
}

void Print(int *a, int n)
{
    static int num = 1;
    cout.width(3);
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, m, *a;
    cout << "N = ";
    cin >> n;
    cout << "M = ";
    cin >> m;
    int h = n > m ? n : m; // размер
    массива a выбирается как max(n,m)
    a = new int[h];
    for (int i = 0; i < h; i++)
        a[i] = 1;
    Print(a, m);
    while (NextSet(a, n, m))
        Print(a, m);
    cin.get(); cin.get();
    return 0;
}
```


Результат работы приведенного выше алгоритма:

C:\Projects\MyProgram\Debug\MyProgram.exe

N = 4

M = 3

```
1:  1 1 1
2:  1 1 2
3:  1 1 3
4:  1 1 4
5:  1 2 1
6:  1 2 2
7:  1 2 3
8:  1 2 4
9:  1 3 1
10: 1 3 2
11: 1 3 3
12: 1 3 4
13: 1 4 1
14: 1 4 2
15: 1 4 3
16: 1 4 4
17: 2 1 1
18: 2 1 2
19: 2 1 3
```

Генерация размещения по номеру

```
function number_to_arrangement(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            t = A(n - length(p) - 1, k - length(p) - 1);  
            if (number < t) {  
                number_to_arrangement(p + c, number);  
                return;  
            }  
            else {  
                number -= t;  
            }  
        }  
    }  
}
```

- Рассчитать коэффициенты $A(n, k)$ можно заранее или использовать динамическое программирование.
- Для хранения уже использованных символов можно использовать массив.

Генерация размещения по номеру

number_to_arrangement([], number)

n = 5

number = 10

k = 3

p = []



used(1)	used(2)	used(3)	used(4)	used(5)
false	false	false	false	false

```
function number_to_arrangement(p, number) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      t = A(n - length(p) - 1, k - length(p) - 1);  
      if (number < t) {  
        number_to_arrangement(p + c, number);  
        return;  
      }  
      else {  
        number -= t;  
      }  
    }  
  }  
}
```

Генерация размещения по номеру

$n = 5$

$\text{number} = 10$

$k = 3$

$p = []$

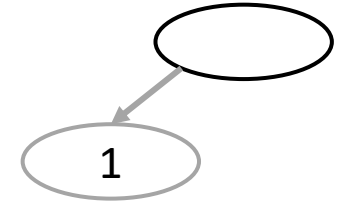
number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number)



used(1)	used(2)	used(3)	used(4)	used(5)
false	false	false	false	false

```
function number_to_arrangement(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            t = A(n - length(p) - 1, k - length(p) - 1);  
            if (number < t) {  
                number_to_arrangement(p + c, number);  
                return;  
            }  
            else {  
                number -= t;  
            }  
        }  
    }  
}
```

Генерация размещения по номеру

$n = 5$

number = 10

$k = 3$

$p = [1]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

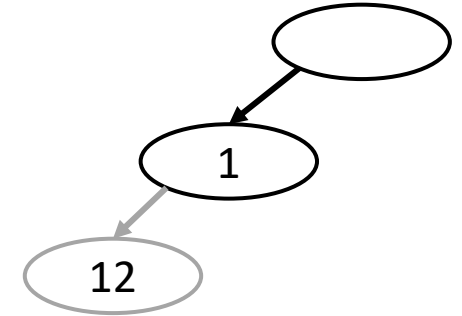
c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

number = $10 - 3 = 7$



```
function number_to_arrangement(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            t = A(n - length(p) - 1, k - length(p) - 1);  
            if (number < t) {  
                number_to_arrangement(p + c, number);  
                return;  
            }  
            else {  
                number -= t;  
            }  
        }  
    }  
}
```

Генерация размещения по номеру

$n = 5$

$\text{number} = 7$

$k = 3$

$p = [1]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```
function number_to_arrangement(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            t = A(n - length(p) - 1, k - length(p) - 1);  
            if (number < t) {  
                number_to_arrangement(p + c, number);  
                return;  
            }  
            else {  
                number -= t;  
            }  
        }  
    }  
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

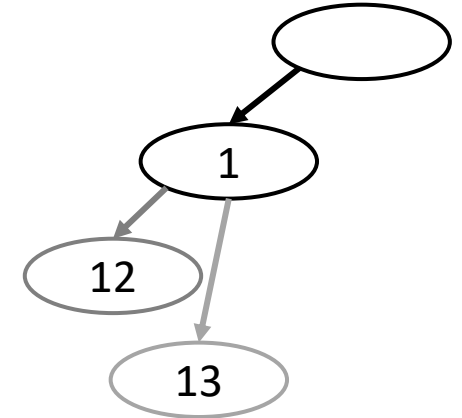
$\text{number} = 10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

$\text{number} = 7 - 3 = 4$



Генерация размещения по номеру

$n = 5$

$\text{number} = 4$

$k = 3$

$p = [1]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

$\text{number} = 10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

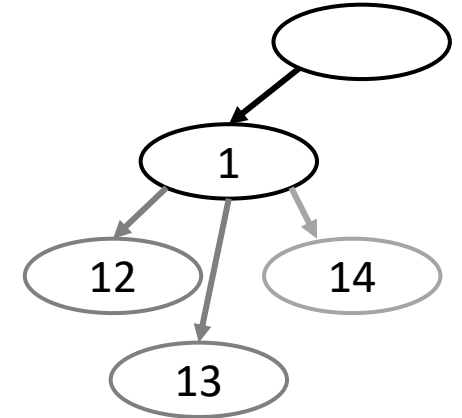
$\text{number} = 7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

$\text{number} = 4 - 3 = 1$



Генерация размещения по номеру

$n = 5$

number = 1

$k = 3$

$p = [1]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

number = $10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

number = $7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

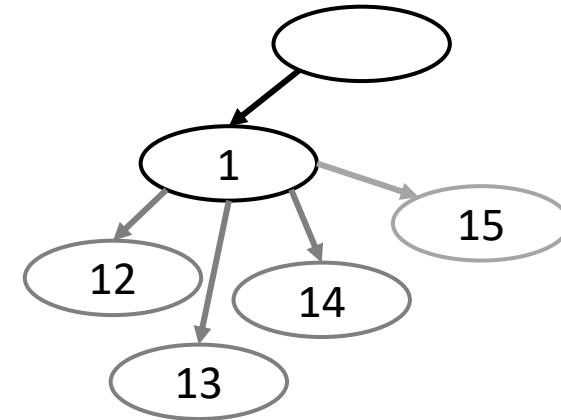
number = $4 - 3 = 1$

c = 5 is not used:

$t = 3$

$1 < 3:$

number_to_arrangement([1,5], number)



Генерация размещения по номеру

$n = 5$

number = 1

$k = 3$

$p = [1, 5]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	true

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

number = $10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

number = $7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

number = $4 - 3 = 1$

c = 5 is not used:

$t = 3$

$1 < 3:$

number_to_arrangement([1, 5], number):

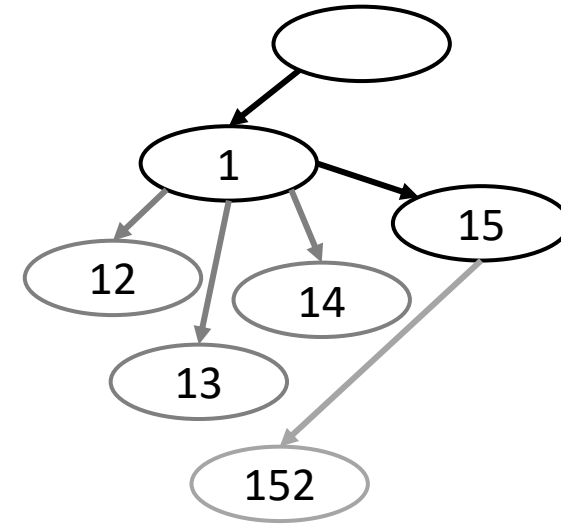
c = 1 is used

c = 2 is not used:

$t = A(5 - 3, 3 - 3) = A(2, 0) = 1$

$1 \geq 1:$

number = $1 - 1 = 0$



Генерация размещения по номеру

$n = 5$

number = 0

$k = 3$

$p = [1, 5]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	true

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

number = $10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

number = $7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

number = $4 - 3 = 1$

c = 5 is not used:

$t = 3$

$1 < 3:$

number_to_arrangement([1,5], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 3, 3 - 3) = A(2, 0) = 1$

$1 \geq 1:$

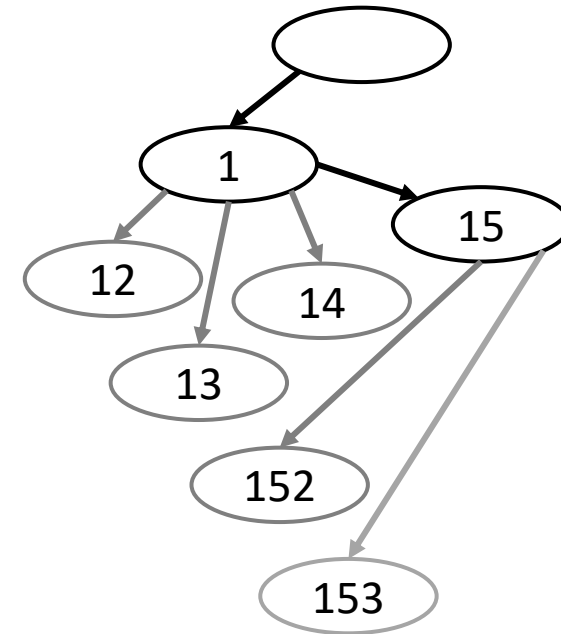
number = $1 - 1 = 0$

c = 3 is not used:

$t = 1$

$0 < 1:$

number_to_arrangement([1,5,3], number)



Генерация размещения по номеру

$n = 5$

number = 0

$k = 3$

$p = [1, 5, 3]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	true	false	true

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

number = $10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

number = $7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

number = $4 - 3 = 1$

c = 5 is not used:

$t = 3$

$1 < 3:$

number_to_arrangement([1,5], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 3, 3 - 3) = A(2, 0) = 1$

$1 \geq 1:$

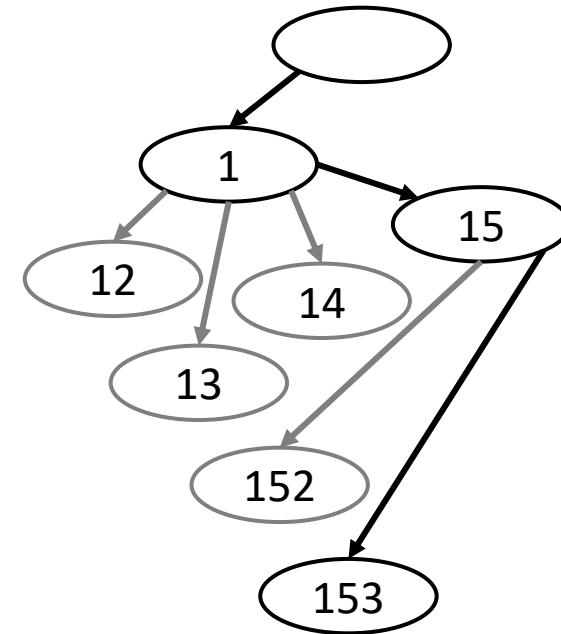
number = $1 - 1 = 0$

c = 3 is not used:

$t = 1$

$0 < 1:$

number_to_arrangement([1,5,3], number)



Генерация размещения по номеру

$n = 5$

$number = 0$

$k = 3$

$p = [1, 5, 3]$

used(1)	used(2)	used(3)	used(4)	used(5)
true	false	true	false	true

```
function number_to_arrangement(p, number) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            t = A(n - length(p) - 1, k - length(p) - 1);
            if (number < t) {
                number_to_arrangement(p + c, number);
                return;
            }
            else {
                number -= t;
            }
        }
    }
}
```

number_to_arrangement([], number):

c = 1 is not used:

$t = A(5 - 1, 3 - 1) = A(4, 2) = 12$

$10 < 12:$

number_to_arrangement([1], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$10 \geq 3:$

$number = 10 - 3 = 7$

c = 3 is not used:

$t = 3$

$7 \geq 3:$

$number = 7 - 3 = 4$

c = 4 is not used:

$t = 3$

$4 \geq 3:$

$number = 4 - 3 = 1$

c = 5 is not used:

$t = 3$

$1 < 3:$

number_to_arrangement([1,5], number):

c = 1 is used

c = 2 is not used:

$t = A(5 - 3, 3 - 3) = A(2, 0) = 1$

$1 \geq 1:$

$number = 1 - 1 = 0$

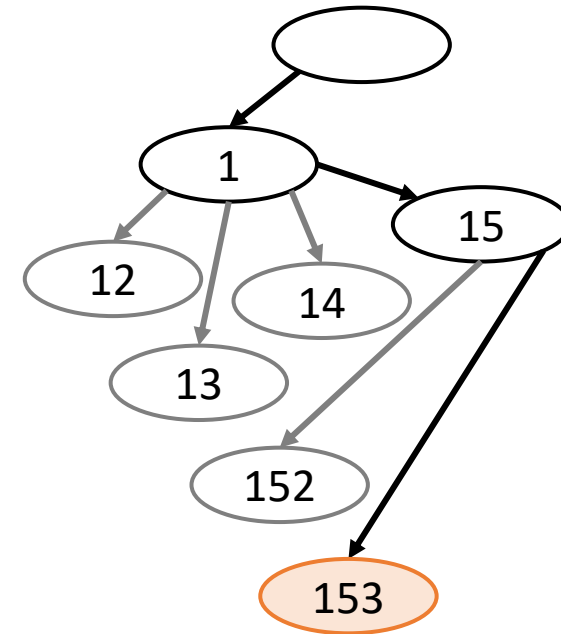
c = 3 is not used:

$t = 1$

$0 < 1:$

number_to_arrangement([1,5,3], number):

$length([1,5,3]) = k = 3$



Получение номера размещения

```
function arrangement_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix([p + c], o)) {  
                arrangement_to_number(p + c, number);  
                return;  
            }  
            else {  
                number += A(n - length(p + c), k - length(p + c));  
            }  
        }  
    }  
}
```

- Рассчитать коэффициенты $A(n,k)$ можно заранее или использовать динамическое программирование.
- Функция `is_prefix(p, o)` сверяет совпадение последнего числа префикса и числа к.о. на той же позиции.

Получение номера размещения

arrangement_to_number([], number, o)

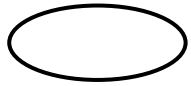
n = 5

k = 3

o = [1,5,3]

number = 0

p = []

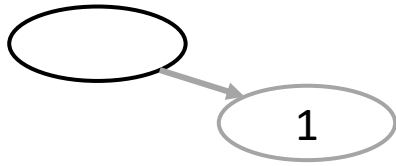


used(1)	used(2)	used(3)	used(4)	used(5)
false	false	false	false	false

```
function arrangement_to_number(p, number, o) {
  if (length(p) == k) {
    print(p);
    return;
  }
  for (c in A) {
    if (not used(c)) {
      if (is_prefix([p + c], o)) {
        arrangement_to_number(p + c, number);
        return;
      }
      else {
        number += A(n - length(p + c), k - length(p + c));
      }
    }
  }
}
```

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 0
p = []



arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

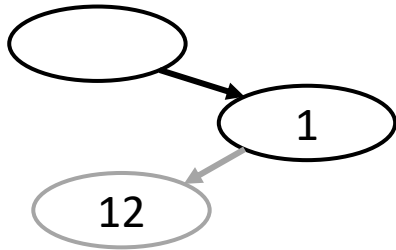
arrangement_to_number([1], number, o)

used(1)	used(2)	used(3)	used(4)	used(5)
false	false	false	false	false

```
function arrangement_to_number(p, number, o) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      if (is_prefix([p + c], o)) {  
        arrangement_to_number(p + c, number);  
        return;  
      }  
      else {  
        number += A(n - length(p + c), k - length(p + c));  
      }  
    }  
  }  
}
```

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 0
p = [1]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

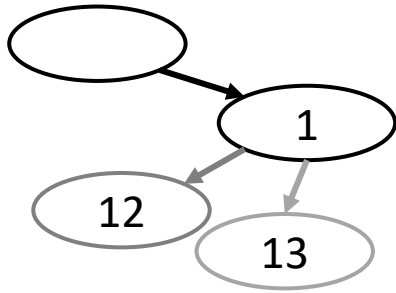
[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

```
function arrangement_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix([p + c], o)) {  
                arrangement_to_number(p + c, number);  
                return;  
            }  
            else {  
                number += A(n - length(p + c), k - length(p + c));  
            }  
        }  
    }  
}
```


Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 3
p = [1]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```
function arrangement_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A) {  
        if (not used(c)) {  
            if (is_prefix([p + c], o)) {  
                arrangement_to_number(p + c, number);  
                return;  
            }  
            else {  
                number += A(n - length(p + c), k - length(p + c));  
            }  
        }  
    }  
}
```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

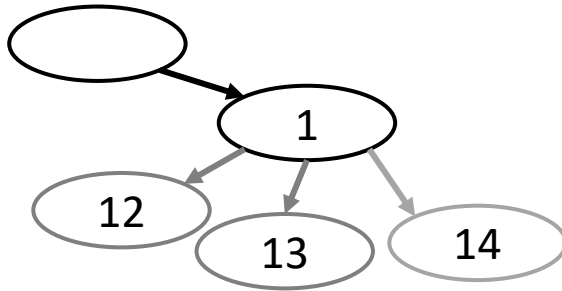
c = 3 is not used:

[1,3] is not prefix:

number = $3 + 3 = 6$

Получение номера размещения

$n = 5$
 $k = 3$
 $o = [1, 5, 3]$
 $number = 6$
 $p = [1]$



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```

function arrangement_to_number(p, number, o) {
  if (length(p) == k) {
    print(p);
    return;
  }
  for (c in A) {
    if (not used(c)) {
      if (is_prefix([p + c], o)) {
        arrangement_to_number(p + c, number);
        return;
      }
      else {
        number += A(n - length(p + c), k - length(p + c));
      }
    }
  }
}
  
```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

$number = A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

$number = 3 + 3 = 6$

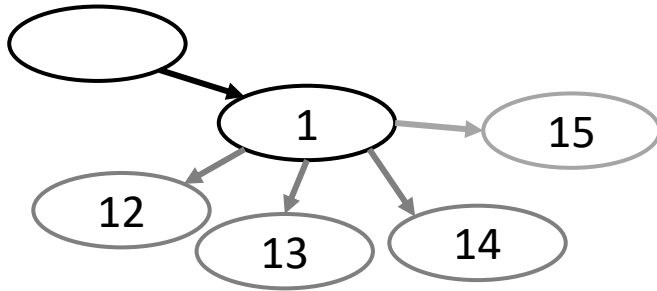
c = 4 is not used:

[1,4] is not prefix:

$number = 6 + 3 = 9$

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 9
p = [1]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	false

```

function arrangement_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            if (is_prefix([p + c], o)) {
                arrangement_to_number(p + c, number);
                return;
            }
            else {
                number += A(n - length(p + c), k - length(p + c));
            }
        }
    }
}

```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

number = $3 + 3 = 6$

c = 4 is not used:

[1,4] is not prefix:

number = $6 + 3 = 9$

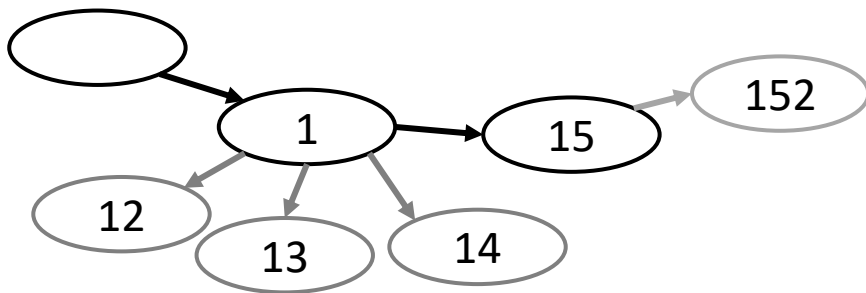
c = 5 is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o)

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 9
p = [1,5]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	true

```

function arrangement_to_number(p, number, o) {
  if (length(p) == k) {
    print(p);
    return;
  }
  for (c in A) {
    if (not used(c)) {
      if (is_prefix([p + c], o)) {
        arrangement_to_number(p + c, number);
        return;
      }
      else {
        number += A(n - length(p + c), k - length(p + c));
      }
    }
  }
}

```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

number = $3 + 3 = 6$

c = 4 is not used:

[1,4] is not prefix:

number = $6 + 3 = 9$

c = 5 is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o):

c = 1 is used

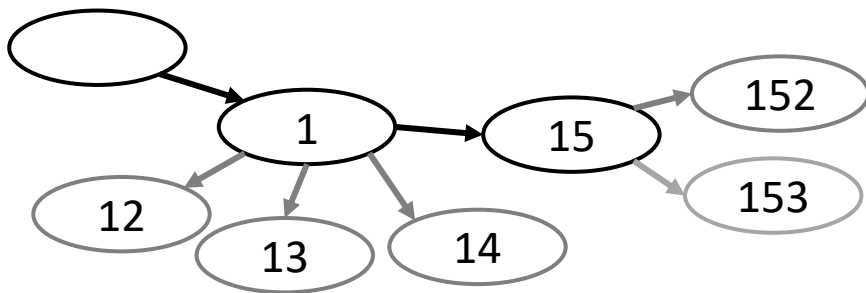
c = 2 is not used:

[1,5,2] is not prefix:

number = $9 + A(5 - 3, 3 - 3) = 10$

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 10
p = [1,5]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	false	false	true

```
function arrangement_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            if (is_prefix([p + c], o)) {
                arrangement_to_number(p + c, number);
                return;
            }
            else {
                number += A(n - length(p + c), k - length(p + c));
            }
        }
    }
}
```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

number = $3 + 3 = 6$

c = 4 is not used:

[1,4] is not prefix:

number = $6 + 3 = 9$

c = 5 is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o):

c = 1 is used

c = 2 is not used:

[1,5,2] is not prefix:

number = $9 + A(5 - 3, 3 - 3) = 10$

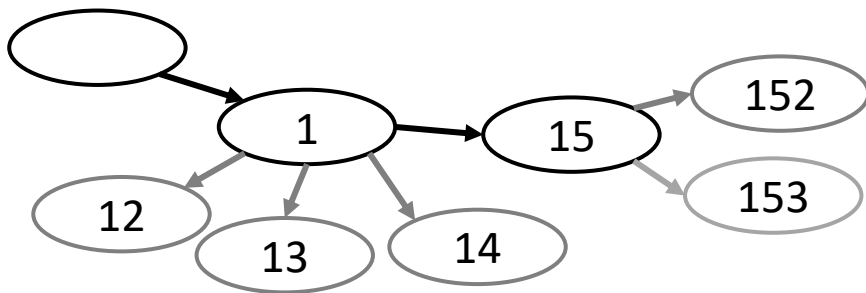
c = 3 is not used:

[1,5,3] is prefix:

arrangement_to_number([1,5,3], number, o)

Получение номера размещения

$n = 5$
 $k = 3$
 $o = [1, 5, 3]$
 $number = 10$
 $p = [1, 5]$



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	true	false	true

```

function arrangement_to_number(p, number, o) {
  if (length(p) == k) {
    print(p);
    return;
  }
  for (c in A) {
    if (not used(c)) {
      if (is_prefix([p + c], o)) {
        arrangement_to_number(p + c, number);
        return;
      }
      else {
        number += A(n - length(p + c), k - length(p + c));
      }
    }
  }
}

```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

$number = A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

$number = 3 + 3 = 6$

c = 4 is not used:

[1,4] is not prefix:

$number = 6 + 3 = 9$

c = 5 is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o):

c = 1 is used

c = 2 is not used:

[1,5,2] is not prefix:

$number = 9 + A(5 - 3, 3 - 3) = 10$

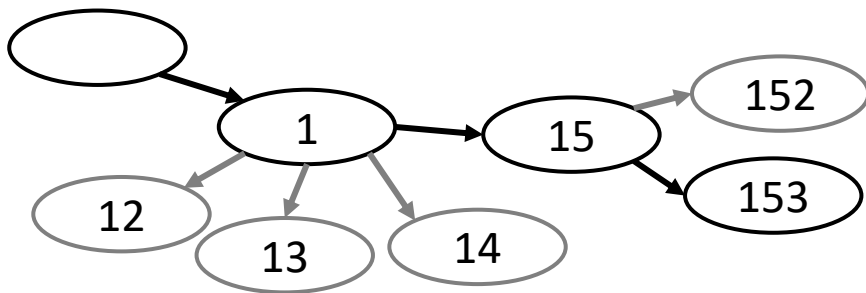
c = 3 is not used:

[1,5,3] is prefix:

arrangement_to_number([1,5,3], number, o)

Получение номера размещения

n = 5
k = 3
o = [1,5,3]
number = 10
p = [1,5,3]



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	true	false	true

```
function arrangement_to_number(p, number, o) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A) {  
    if (not used(c)) {  
      if (is_prefix([p + c], o)) {  
        arrangement_to_number(p + c, number);  
        return;  
      }  
      else {  
        number += A(n - length(p + c), k - length(p + c));  
      }  
    }  
  }  
}
```

arrangement_to_number([], number, o):

c = 1 is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

c = 1 is used

c = 2 is not used:

[1,2] is not prefix:

number = $A(5 - 2, 3 - 2) = A(3, 1) = 3$

c = 3 is not used:

[1,3] is not prefix:

number = $3 + 3 = 6$

c = 4 is not used:

[1,4] is not prefix:

number = $6 + 3 = 9$

c = 5 is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o):

c = 1 is used

c = 2 is not used:

[1,5,2] is not prefix:

number = $9 + A(5 - 3, 3 - 3) = 10$

c = 3 is not used:

[1,5,3] is prefix:

arrangement_to_number([1,5,3], number, o):

length([1,5,3]) = k = 3

Получение номера размещения

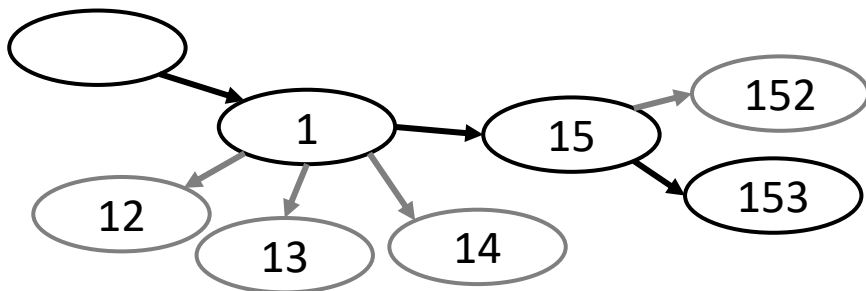
$n = 5$

$k = 3$

$o = [1, 5, 3]$

$number = 10$

$p = [1, 5, 3]$



used(1)	used(2)	used(3)	used(4)	used(5)
true	false	true	false	true

```
function arrangement_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A) {
        if (not used(c)) {
            if (is_prefix([p + c], o)) {
                arrangement_to_number(p + c, number);
                return;
            }
            else {
                number += A(n - length(p + c), k - length(p + c));
            }
        }
    }
}
```

arrangement_to_number([], number, o):

$c = 1$ is not used:

[1] is prefix:

arrangement_to_number([1], number, o):

$c = 1$ is used

$c = 2$ is not used:

[1,2] is not prefix:

$number = A(5 - 2, 3 - 2) = A(3, 1) = 3$

$c = 3$ is not used:

[1,3] is not prefix:

$number = 3 + 3 = 6$

$c = 4$ is not used:

[1,4] is not prefix:

$number = 6 + 3 = 9$

$c = 5$ is prefix:

[1,5] is prefix:

arrangement_to_number([1,5], number, o):

$c = 1$ is used

$c = 2$ is not used:

[1,5,2] is not prefix:

$number = 9 + A(5 - 3, 3 - 3) = 10$

$c = 3$ is not used:

[1,5,3] is prefix:

arrangement_to_number([1,5,3], number, o):

$length([1,5,3]) = k = 3$

Алгоритм порождения сочетаний в лексикографическом порядке

- Лексикографический порядок: в каждом сочетании элементы расположены в порядке возрастания слева направо. Например, сочетания из шести элементов (N) по три (k) в возрастающем лексикографическом порядке :

123	↓	135	↓	234	↓	256	↓
124		136		235		345	
125		145		236		346	
126		146		245		356	
134		156		246		456	

Всего 20 разных сочетаний из 6 по 3 элемента

Алгоритм порождения сочетаний в лексикографическом порядке

- Начинаем с сочетания $(1, 2, \dots, k)$, (123)
- просматриваем текущее сочетание справа налево, определяем место самого правого элемента, который не достиг еще своего максимального значения. (123)
- Этот элемент увеличиваем на единицу, и всем элементам справа от него присваиваем новые *наименьшие возможные значения* (124)

126 $\rightarrow$ 134,
почему 4? т.к. $4 > 3$

Почему в каждом сочетании элементы также расположены по возрастанию?

123
124
125
126 246
134 256
135 345
136 346
145 356
146 456
156
234
235
236
245

Алгоритм порождения сочетаний в лексикографическом порядке

```
for i ← 1 to k do
    c[i] ← i
j ← 1
while j != 0 do
    { вывести (c[1], c[2], ... , c[k]) }
    j ← k
    while c[j] >= n - k + j do
        j ← (j - 1)
    c[j] ← c[j] + 1
    for i = j + 1 to k do
        c[i] ← c[i - 1] + 1
}
```

- Начальное сочетание $1, 2, \dots, k$
- Последнее сочетание $N - k + 1, N - k + 2, \dots, N$
- Переход от текущего сочетания к следующему – Просматриваем сочетание справа налево
- Находим первый элемент, который можно увеличить
- Увеличиваем его на единицу
- Часть сочетания от этого элемента до конца формируем из натуральных чисел, следующих за ним
- Какой элемент можно увеличить в сочетании? Если $N=5, k=3$, то максимальное лексикографическое сочетание 345. Значит, на месте k может стоять максимальный элемент N , на месте $(k-1)$ может стоять максимальный элемент $(N-1)$.
- На месте i может стоять максимальный элемент $(N - k + i)$.

Алгоритм порождения сочетаний в лексикографическом порядке

```
GetNext;  
  i ← k;  
  while (C[i] + k - i + 1 > N) do i--;  
    //Находим элемент, который можно увеличить  
  C[i]++; //Увеличиваем на единицу  
  for j ← i + 1 to k do  
    C[j] ← C[j - 1] + 1;  
    //Изменяем стоящие справа элементы  
end;
```

Алгоритм порождения сочетаний в лексикографическом порядке

```
bool NextSet(int *a, int n, int m)
{
    int k = m;
    for (int i = k - 1; i >= 0; --i)
        if (a[i] < n - k + i + 1)
        {
            ++a[i];
            for (int j = i + 1; j < k; ++j)
                a[j] = a[j - 1] + 1;
            return true;
        }
    return false;
}

void Print(int *a, int n)
{
    static int num = 1;
    cout.width(3);
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, m, *a;
    cout << "N = ";
    cin >> n;
    cout << "M = ";
    cin >> m;
    a = new int[n];
    for (int i = 0; i < n; i++)
        a[i] = i + 1;
    Print(a, m);
    if (n >= m)
    {
        while (NextSet(a, n, m))
            Print(a, m);
    }
    cin.get(); cin.get();
    return 0;
}
```

Результат выполнения

C:\Projects\MyProgram\Debug\MyProgram.exe

N = 6

M = 4

```
1: 1 2 3 4
2: 1 2 3 5
3: 1 2 3 6
4: 1 2 4 5
5: 1 2 4 6
6: 1 2 5 6
7: 1 3 4 5
8: 1 3 4 6
9: 1 3 5 6
10: 1 4 5 6
11: 2 3 4 5
12: 2 3 4 6
13: 2 3 5 6
14: 2 4 5 6
15: 3 4 5 6
```

Алгоритм порождения сочетаний с повторениями

```
bool NextSet(int *a, int n, int m)
{
    int j = m - 1;
    while (a[j] == n && j >= 0) j--;
    if (j < 0) return false;
    if (a[j] >= n)
        j--;
    a[j]++;
    if (j == m - 1) return true;
    for (int k = j + 1; k < m; k++)
        a[k] = a[j];
    return true;
}

void Print(int *a, int n)
{
    static int num = 1;
    cout.width(3);
    cout << num++ << ": ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
int main()
{
    int n, m, *a;
    cout << "N = ";
    cin >> n;
    cout << "M = ";
    cin >> m;
    int h = n > m ? n : m; // размер
    массива a выбирается как max(n,m)
    a = new int[h];
    for (int i = 0; i < h; i++)
        a[i] = 1;
    Print(a, m);
    while (NextSet(a, n, m))
        Print(a, m);
    cin.get(); cin.get();
    return 0;
}
```

Результат работы приведенного выше алгоритма:

C:\Projects\MyProgram\Debug\MyProgram.exe

N = 4

M = 3

1: 1 1 1

2: 1 1 2

3: 1 1 3

4: 1 1 4

5: 1 2 2

6: 1 2 3

7: 1 2 4

8: 1 3 3

9: 1 3 4

10: 1 4 4

11: 2 2 2

12: 2 2 3

13: 2 2 4

14: 2 3 3

15: 2 3 4

16: 2 4 4

17: 3 3 3

18: 3 3 4

19: 3 4 4

20: 4 4 4

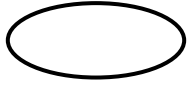
Генерация сочетания по номеру

```
function number_to_combination(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
        t = binomial(n - c, k - length(p) - 1)  
        if (number < t) {  
            number_to_combination(p + c, number);  
            return;  
        }  
        else {  
            number -= t;  
        }  
    }  
}
```

- Рассчитать биномиальные коэффициенты можно заранее или использовать динамическое программирование.

Генерация сочетаний по номеру

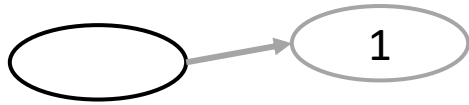
number_to_combination([], 5)



n = 5
k = 3
number = 5
p = []

```
function number_to_combination(p, number) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
    t = binomial(n - c, k - length(p) - 1)  
    if (number < t) {  
      number_to_combination(p + c, number);  
      return;  
    }  
    else {  
      number -= t;  
    }  
  }  
}
```

Генерация сочетаний по номеру



$n = 5$
 $k = 3$
 $\text{number} = 5$
 $p = []$

number_to_combination([], 5):

$A = \{1, 2, 3, 4, 5\}$

$c = 1$:

$t = \text{binomial}(5 - 1, 3 - 1) = \text{binomial}(4, 2) = 6$

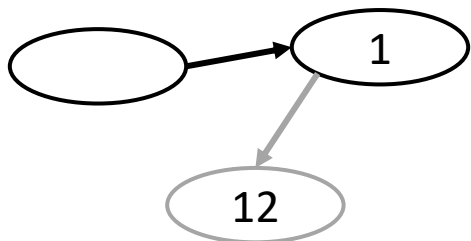
$5 < 6$:

number_to_combination([1], number)

```
function number_to_combination(p, number) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
    t = binomial(n - c, k - length(p) - 1)  
    if (number < t) {  
      number_to_combination(p + c, number);  
      return;  
    }  
    else {  
      number -= t;  
    }  
  }  
}
```

Генерация сочетаний по номеру

$n = 5$
 $k = 3$
 $\text{number} = 5$
 $p = [1]$



```
function number_to_combination(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
        t = binomial(n - c, k - length(p) - 1)  
        if (number < t) {  
            number_to_combination(p + c, number);  
            return;  
        }  
        else {  
            number -= t;  
        }  
    }  
}
```

number_to_combination([], 5):

$A = \{1, 2, 3, 4, 5\}$

$c = 1$:

$t = \text{binomial}(5 - 1, 3 - 1) = \text{binomial}(4, 2) = 6$

$5 < 6$:

number_to_combination([1], number):

$A = \{2, 3, 4\}$

$c = 2$:

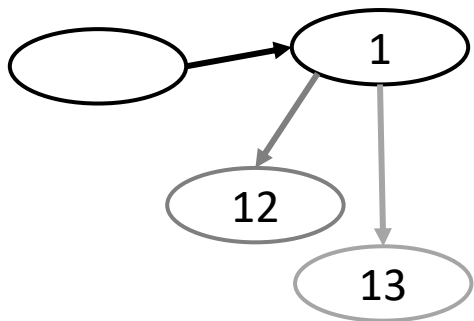
$t = \text{binomial}(5 - 2, 3 - 2) = \text{binomial}(3, 1) = 3$

$5 \geq 3$:

$\text{number} = 5 - 3 = 2$

Генерация сочетаний по номеру

$n = 5$
 $k = 3$
 $\text{number} = 2$
 $p = [1]$



```
function number_to_combination(p, number) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
    t = binomial(n - c, k - length(p) - 1)  
    if (number < t) {  
      number_to_combination(p + c, number);  
      return;  
    }  
    else {  
      number -= t;  
    }  
  }  
}
```

number_to_combination([], 5):

$A = \{1, 2, 3, 4, 5\}$

$c = 1$:

$t = \text{binomial}(5 - 1, 3 - 1) = \text{binomial}(4, 2) = 6$

$5 < 6$:

number_to_combination([1], number):

$A = \{2, 3, 4\}$

$c = 2$:

$t = \text{binomial}(5 - 2, 3 - 2) = \text{binomial}(3, 1) = 3$

$5 \geq 3$:

$\text{number} = 5 - 3 = 2$

$c = 3$:

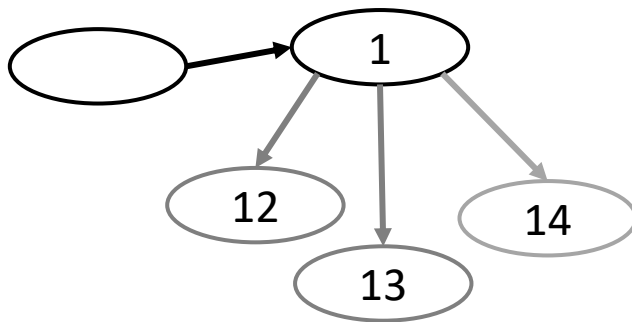
$t = \text{binomial}(5 - 3, 3 - 2) = \text{binomial}(2, 1) = 2$

$2 \geq 2$:

$\text{number} = 2 - 2 = 0$

Генерация сочетаний по номеру

$n = 5$
 $k = 3$
 $\text{number} = 0$
 $p = [1]$



```
function number_to_combination(p, number) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
        t = binomial(n - c, k - length(p) - 1)  
        if (number < t) {  
            number_to_combination(p + c, number);  
            return;  
        }  
        else {  
            number -= t;  
        }  
    }  
}
```

number_to_combination([], 5):

$A = \{1, 2, 3, 4, 5\}$

$c = 1$:

$t = \text{binomial}(5 - 1, 3 - 1) = \text{binomial}(4, 2) = 6$

$5 < 6$:

number_to_combination([1], number):

$A = \{2, 3, 4\}$

$c = 2$:

$t = \text{binomial}(5 - 2, 3 - 2) = \text{binomial}(3, 1) = 3$

$5 \geq 3$:

$\text{number} = 5 - 3 = 2$

$c = 3$:

$t = \text{binomial}(5 - 3, 3 - 2) = \text{binomial}(2, 1) = 2$

$2 \geq 2$:

$\text{number} = 2 - 2 = 0$

$c = 4$:

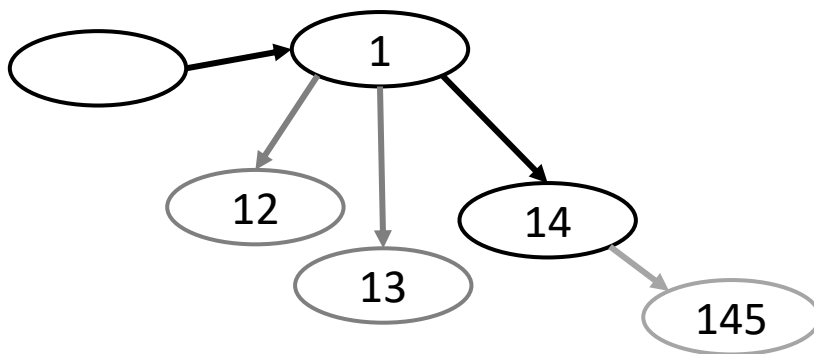
$t = \text{binomial}(5 - 4, 3 - 2) = \text{binomial}(1, 1) = 1$

$0 < 1$:

number_to_combination([1,4], number)

Генерация сочетаний по номеру

$n = 5$
 $k = 3$
 $\text{number} = 0$
 $p = [1, 4]$



```
function number_to_combination(p, number) {  
  if (length(p) == k) {  
    print(p);  
    return;  
  }  
  for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
    t = binomial(n - c, k - length(p) - 1)  
    if (number < t) {  
      number_to_combination(p + c, number);  
      return;  
    }  
    else {  
      number -= t;  
    }  
  }  
}
```

number_to_combination([], 5):

$A = \{1, 2, 3, 4, 5\}$

$c = 1$:

$t = \text{binomial}(5 - 1, 3 - 1) = \text{binomial}(4, 2) = 6$

$5 < 6$:

number_to_combination([1], number):

$A = \{2, 3, 4\}$

$c = 2$:

$t = \text{binomial}(5 - 2, 3 - 2) = \text{binomial}(3, 1) = 3$

$5 \geq 3$:

$\text{number} = 5 - 3 = 2$

$c = 3$:

$t = \text{binomial}(5 - 3, 3 - 2) = \text{binomial}(2, 1) = 2$

$2 \geq 2$:

$\text{number} = 2 - 2 = 0$

$c = 4$:

$t = \text{binomial}(5 - 4, 3 - 2) = \text{binomial}(1, 1) = 1$

$0 < 1$:

number_to_combination([1, 4], number):

$A = \{5\}$

$c = 5$:

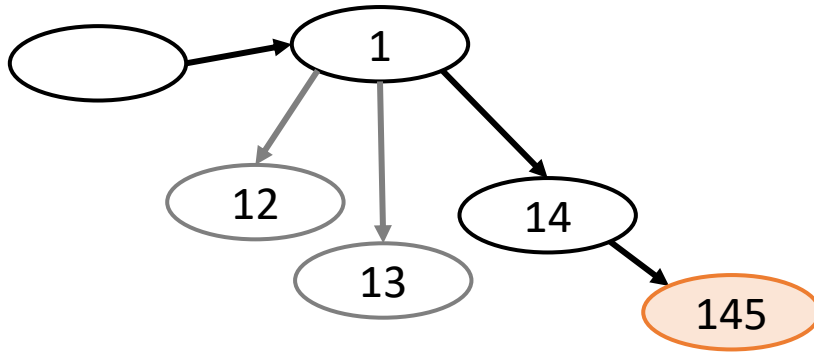
$t = \text{binomial}(5 - 5, 3 - 3) = \text{binomial}(0, 0) = 1$

$0 < 1$:

number_to_combination([1, 4, 5], number)

Генерация сочетаний по номеру

n = 5
k = 3
number = 0
p = [1,4,5]



```

function number_to_combination(p, number) {
  if (length(p) == k) {
    print(p);
    return;
  }
  for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {
    t = binomial(n - c, k - length(p) - 1)
    if (number < t) {
      number_to_combination(p + c, number);
      return;
    }
    else {
      number -= t;
    }
  }
}

```

number_to_combination([], 5):

A = {1, 2, 3, 4, 5}

c = 1:

t = binomial(5 - 1, 3 - 1) = binomial(4, 2) = 6

5 < 6:

number_to_combination([1], number):

A = {2, 3, 4}

c = 2:

t = binomial(5 - 2, 3 - 2) = binomial(3, 1) = 3

5 >= 3:

number = 5 - 3 = 2

c = 3:

t = binomial(5 - 3, 3 - 2) = binomial(2, 1) = 2

2 >= 2:

number = 2 - 2 = 0

c = 4:

t = binomial(5 - 4, 3 - 2) = binomial(1, 1) = 1

0 < 1:

number_to_combination([1,4], number):

A = {5}

c = 5:

t = binomial(5 - 5, 3 - 3) = binomial(0, 0) = 1

0 < 1:

number_to_combination([1,4,5], number):

length([1,4,5]) = k = 3

Получение номера сочетания

```
function object_to_number(p, number, o) {  
    if (p – комбинаторный объект) { префикс является сочетанием, если его длина равна k  
        print(number);  
        return;  
    }  
    for (c in A) { перебор сокращается с каждым вызовом: он начинается с последнего числа префикса, увеличенного на 1 или 1,  
        if ([p + c] – префикс комбинаторного объекта) { если префикс пока пустой, и заканчивается максимально возможным числом, доступным для данного префикса  
            if ([p + c] входит в нужную ветку) { особых требований к префиксу не предъявляется, они учтены ранее  
                object_to_number(p + c, number, o); условие равносильно тому, что [p + c] является префиксом o  
                return;  
            }  
            else {  
                number += количество к.о. с префиксом [p + c];  $C(n - c, k - \text{length}(p) - 1)$   
            }  
        }  
    }  
}
```

Получение номера сочетания

```
function combination_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k + length(p) + 1}) {  
        if (is_prefix([p + c], o)) {  
            combination_to_number(p + c, number);  
            return;  
        }  
        else {  
            number += binomial(n - c, k - length(p) - 1);  
        }  
    }  
}
```

- Рассчитать биномиальные коэффициенты можно заранее или использовать динамическое программирование.
- Функция `is_prefix(p, o)` сверяет совпадение последнего числа префикса и числа к.о. на той же позиции.

Получение номера сочетания

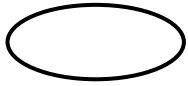
n = 5

k = 3

o = [1,4,5]

number = 0

p = []

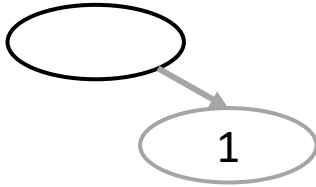


combination_to_number([], number, o)

```
function combination_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A = {last_digit(p) + 1, ..., n - k +
length(p) + 1}) {
        if (is_prefix([p + c], o)) {
            combination_to_number(p + c, number);
            return;
        }
        else {
            number += binomial(n - c, k - length(p)
- 1);
        }
    }
}
```

Получение номера сочетания

n = 5
k = 3
o = [1,4,5]
number = 0
p = []



combination_to_number([], number, o):

A = {1,2,3}

c = 1:

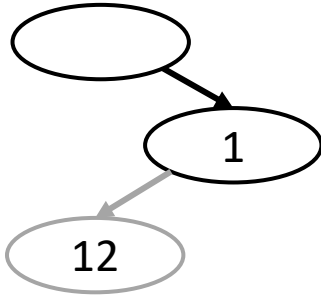
[1] is prefix:

combination_to_number([1], number, o)

```
function combination_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k +  
length(p) + 1}) {  
        if (is_prefix([p + c], o)) {  
            combination_to_number(p + c, number);  
            return;  
        }  
        else {  
            number += binomial(n - c, k - length(p)  
- 1);  
        }  
    }  
}
```

Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 0$
 $p = [1]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

$[1]$ is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$

$c = 2$:

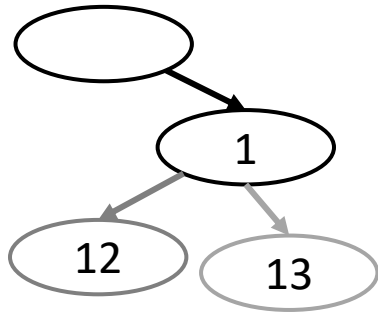
$[1, 2]$ is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

```
function combination_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k +  
length(p) + 1}) {  
        if (is_prefix([p + c], o)) {  
            combination_to_number(p + c, number);  
            return;  
        }  
        else {  
            number += binomial(n - c, k - length(p)  
- 1);  
        }  
    }  
}
```

Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 3$
 $p = [1]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

$[1]$ is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$:

$c = 2$:

$[1, 2]$ is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

$c = 3$:

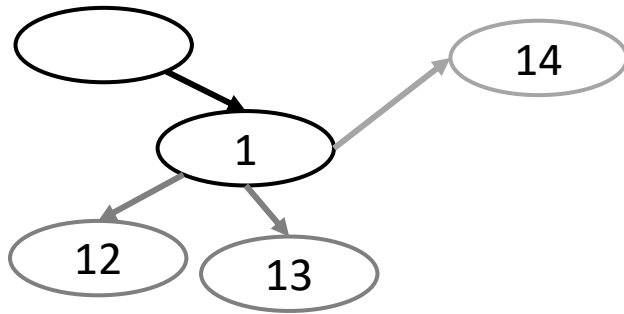
$[1, 3]$ is not prefix:

$number = 3 + \text{binomial}(5 - 3, 3 - 2) = 3 + 2 = 5$

```
function combination_to_number(p, number, o) {  
    if (length(p) == k) {  
        print(p);  
        return;  
    }  
    for (c in A = {last_digit(p) + 1, ..., n - k +  
length(p) + 1}) {  
        if (is_prefix([p + c], o)) {  
            combination_to_number(p + c, number);  
            return;  
        }  
        else {  
            number += binomial(n - c, k - length(p)  
- 1);  
        }  
    }  
}
```

Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 5$
 $p = [1]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

$[1]$ is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$:

$c = 2$:

$[1, 2]$ is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

$c = 3$:

$[1, 3]$ is not prefix:

$number = 3 + \text{binomial}(5 - 3, 3 - 2) = 3 + 2 = 5$

$c = 4$:

$[1, 4]$ is prefix:

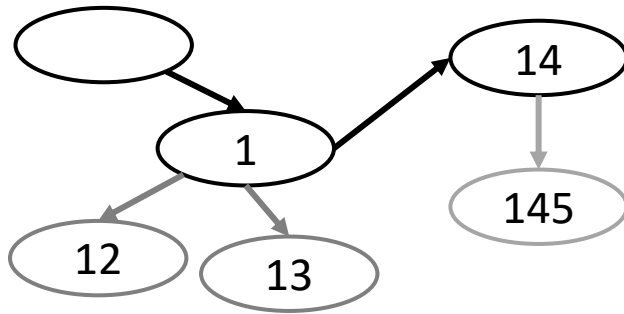
combination_to_number([1, 4], number, o)

```

function combination_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A = {last_digit(p) + 1, ..., n - k +
length(p) + 1}) {
        if (is_prefix([p + c], o)) {
            combination_to_number(p + c, number);
            return;
        }
        else {
            number += binomial(n - c, k - length(p)
- 1);
        }
    }
}
    
```

Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 5$
 $p = [1, 4]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

[1] is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$:

$c = 2$:

[1, 2] is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

$c = 3$:

[1, 3] is not prefix:

$number = 3 + \text{binomial}(5 - 3, 3 - 2) = 3 + 2 = 5$

$c = 4$:

[1, 4] is prefix:

combination_to_number([1, 4], number, o):

$A = \{5\}$

$c = 5$:

[1, 4, 5] is prefix:

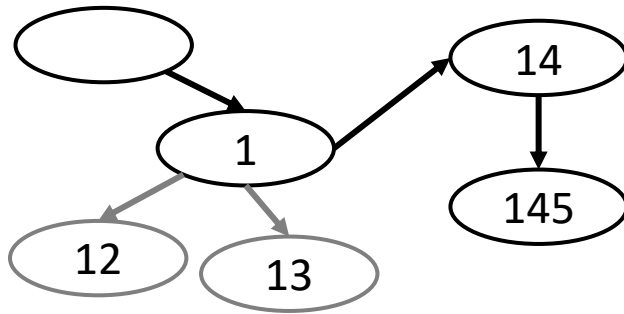
combination_to_number([1, 4, 5], number, o)

```

function combination_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A = {last_digit(p) + 1, ..., n - k +
length(p) + 1}) {
        if (is_prefix([p + c], o)) {
            combination_to_number(p + c, number);
            return;
        }
        else {
            number += binomial(n - c, k - length(p)
- 1);
        }
    }
}
    
```


Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 5$
 $p = [1, 4, 5]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

[1] is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$:

$c = 2$:

[1,2] is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

$c = 3$:

[1,3] is not prefix:

$number = 3 + \text{binomial}(5 - 3, 3 - 2) = 3 + 2 = 5$

$c = 4$:

[1,4] is prefix:

combination_to_number([1,4], number, o):

$A = \{5\}$

$c = 5$:

[1,4,5] is prefix:

combination_to_number([1,4,5], number, o):

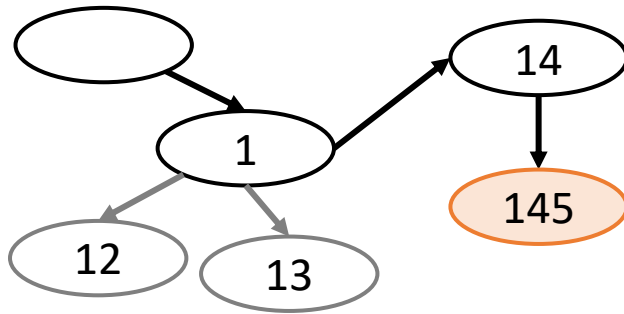
$\text{length}([1,4,5]) = k = 3$

```

function combination_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A = {last_digit(p) + 1, ..., n - k +
length(p) + 1}) {
        if (is_prefix([p + c], o)) {
            combination_to_number(p + c, number);
            return;
        }
        else {
            number += binomial(n - c, k - length(p)
- 1);
        }
    }
}
  
```

Получение номера сочетания

$n = 5$
 $k = 3$
 $o = [1, 4, 5]$
 $number = 5$
 $p = [1, 4, 5]$



combination_to_number([], number, o):

$A = \{1, 2, 3\}$

$c = 1$:

$[1]$ is prefix:

combination_to_number([1], number, o):

$A = \{2, 3, 4\}$:

$c = 2$:

$[1, 2]$ is not prefix:

$number = 0 + \text{binomial}(5 - 2, 3 - 2) = 3$

$c = 3$:

$[1, 3]$ is not prefix:

$number = 3 + \text{binomial}(5 - 3, 3 - 2) = 3 + 2 = 5$

$c = 4$:

$[1, 4]$ is prefix:

combination_to_number([1, 4], number, o):

$A = \{5\}$

$c = 5$:

$[1, 4, 5]$ is prefix:

combination_to_number([1, 4, 5], number, o):

$\text{length}([1, 4, 5]) = k = 3$

```

function combination_to_number(p, number, o) {
    if (length(p) == k) {
        print(p);
        return;
    }
    for (c in A = {last_digit(p) + 1, ..., n - k +
length(p) + 1}) {
        if (is_prefix([p + c], o)) {
            combination_to_number(p + c, number);
            return;
        }
        else {
            number += binomial(n - c, k - length(p)
- 1);
        }
    }
}
    
```

Генерация перестановок, сочетаний, размещений

Вычислительная сложность алгоритмов – $O(n)$, где n – это количество комбинаторных объектов данного размера, так как нужно перебрать все объекты, пока алгоритм не дойдет до нужного. В этом заключается главный недостаток алгоритмов – его неэффективность. Однако, они реализуются достаточно просто.

Генерация разбиений. Алгоритм Гинденбурга

В общем случае шаги алгоритма Гинденбурга начинаются с разбиения, которое состоит только из одного слагаемого, равного n ($p_1=n$). У последующих разбиений число слагаемых m не убывает, а сами слагаемые записываются в неубывающем порядке своих величин. При этом каждое очередное разбиение на m слагаемых строится по текущему разбиению следующим образом. Нужно просмотреть текущее разбиение справа налево, с целью найти наибольшее слагаемое p_i , которое отличается от последнего слагаемого p_m не меньше, чем на 2 :

$$p_m - p_i \geq 2.$$

Слагаемое p_i нужно увеличить на 1 и присвоить его новое значение всем слагаемым p_j , справа от него до предпоследнего слагаемого, включительно. Указанное преобразование текущего разбиения можно формально записать следующим образом:

$$p_j = p_i + 1 \text{ для всех } j = i, \dots, (m - 1).$$

С учетом полученных значений последнее слагаемое p_m вычисляется по следующей остаточной формуле:

$$p_m = n - (p_1 + \dots + p_j + \dots + p_{m-1}).$$

В следующем примере приведены все последовательные разбиения числа 7 на 4 слагаемых, которые получены таким способом, а образующее слагаемое 1 выделено подчеркиванием:

$$1 + 1 + \underline{1} + 4 \rightarrow 1 + \underline{1} + 2 + 3 \rightarrow 1 + 2 + 2 + 2.$$

Генерация разбиений. Алгоритм Гинденбурга

В общем случае каждая серия из последовательных разбиений на m слагаемых завершается лексиграфически наибольшим разбиением, где последнее слагаемое p_m не превосходит по величине остальные слагаемые больше, чем на 1 . Так как слагаемые разбиений записываются в неубывающем порядке своих величин, то такое соотношение их значений отражает следующее неравенство:

$$p_m - p_1 \leq 1.$$

В этом случае нужно перейти к построению следующей серии лексиграфически упорядоченных разбиений на $(m+1)$ слагаемых, где им присваиваются следующие начальные значения:

$$p_1 = 1, \dots, p_m = 1, p_{m+1} = (n - m).$$

Указанный переход показан в следующем примере для последовательных разбиений числа 7 на 4 и 5 слагаемых:

$$1 + 2 + 2 + 2 \rightarrow 1 + 1 + 1 + 1 + 3.$$

Таким образом, на каждом шаге алгоритма Гинденбурга либо по текущему разбиению строится следующее в лексиграфическом порядке разбиение с тем же количеством слагаемых, либо их число увеличивается на 1 и формируется опорное разбиение, где значения всех слагаемых, кроме последнего, равны 1 . Этот процесс генерации разбиений должен быть завершен, когда на конечном шаге получится разбиение с максимальным числом частей, где все значения равны 1 :

$$n = (p_1 = 1) + \dots + (p_i = 1) + \dots + (p_n = 1).$$

Генерация разбиений. Алгоритм Гинденбурга

Можно заметить, что для любого целого числа n последовательность разбиений, формируемых алгоритмом Гинденбурга, частично упорядочена по количеству слагаемых от **1** до n . Поэтому разбиения равной длины всегда оказываются рядом. Это можно использовать для перечисления всех разбиений любого целого числа n , где количество слагаемых фиксировано или ограничено заданным значением $m < n$. Интересно, что величины наибольших слагаемых их сопряженных разбиений оказываются равны или ограничены значением m . Это предоставляет возможность перечислять разбиения, где величина слагаемых ограничена заданным значением.

Генерация разбиений. Алгоритм Эрлиха

Не менее гибкий, но гораздо более совершенный в вычислительном отношении *алгоритм Эрлиха* порождает все разбиения любого заданного числа в словарном порядке, используя мультимножественную запись слагаемых, где учитывается их кратность. Это дает возможность записать разбиение любого числа n на $m \leq n$ мультислагаемых в следующем формате:

$$n = k_1 p_1 + \dots + k_i p_i + \dots + k_m p_m .$$

В этой записи k_i обозначает кратность слагаемого p_i , а все слагаемые перечислены в порядке убывания их величин:

$$p_1 > \dots > p_i > \dots > p_m .$$

Например, разбиение числа **7** на **2** мультислагаемых **2** и **1** с кратностями **2** и **3**, соответственно, выглядит так:

$$7 = 2 \cdot 2 + 3 \cdot 1 = (2 + 2) + (1 + 1 + 1).$$

Мультимножественная запись слагаемых позволяет построить алгоритм генерации разбиений целых чисел, который переходит от текущего разбиения к следующему в словарном порядке, рассматривая только самое правое мультислагаемое без анализа предыдущих частей разбиения как было в алгоритме Гинденбурга. Способ конструирования очередного разбиения зависит от кратности последнего мультислагаемого текущего разбиения. При этом требуется рассмотреть следующие два случая:

$$k_m > 1 \text{ и } k_m = 1.$$

Генерация разбиений. Алгоритм Эрлиха

В первом случае нужно исключить из разбиения значение $(2 \cdot p_m)$, чтобы получить возможность добавить (еще) одно мультислагаемое со значением $(p_m + 1)$. Во втором случае нужно добавить еще одно мультислагаемое со значением $(p_m - 1 + 1)$, исключив два последних мультислагаемых. В любом случае, то, что остается от уменьшаемых мультислагаемых должно быть превращено в соответствующее число единиц для последнего мультислагаемого следующего разбиения. Этот процесс должен начинаться с разбиения, где все части равны единице, и завершаться разбиением из одного мультислагаемого.

Генерация разбиений. Алгоритм Эрлиха

мультиразбиение	исключить	добавить	остаток
$7 \cdot 1 = 1+1+1+1+1+1+1$	2·1	1·2	5·1
$1 \cdot 2 + 5 \cdot 1 = 2+(1+1+1+1+1)$	2·1	1·2	3·1
$2 \cdot 2 \ 3 \cdot 1 = (2+2)+(1+1+1)$	2·1	1·2	1·1
$3 \cdot 2 + 1 \cdot 1 = (2+2+2)+1$	3·2, 11	1·3	4·1
$1 \cdot 3 + 4 \cdot 1 = 3+(1+1+1+1)$	2·1	1·2	2·1
$1 \cdot 3 + 1 \cdot 2 + 2 \cdot 1 = 3+2+(1+1)$	2·1	1·2	0
$1 \cdot 3 + 2 \cdot 2 = 3+(2+2)$	2·2	1·3	1·1
$2 \cdot 3 + 1 \cdot 1 = (3+3)+1$	2·2, 1·1	1·4	3·1
$1 \cdot 4 + 3 \cdot 1 = 4+(1+1+1)$	2·1	1·2	1·1
$1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 4+2+1$	1·2, 1·1	1·3	0
$1 \cdot 4 + 1 \cdot 3 = 4 + 3$	1·4, 1·3	1·5	2·1
$1 \cdot 5 + 2 \cdot 1 = 5+(1+1)$	2·1	1·2	0
$1 \cdot 5 + 1 \cdot 2 = 5+2$	1·5, 1·2	1·6	1·1
$1 \cdot 6 + 1 \cdot 1 = 6+1$	1·6, 1·1	1·7	0
$1 \cdot 7 = 7$	-	-	-

Алгоритм Гинденбурга. Алгоритм Эрлиха

Алгоритм Гинденбурга - алгоритм, порождающий разбиения любых натуральных чисел в порядке увеличения количества слагаемых, а разбиения равной длины перечисляются в лексиграфическом порядке. Алгоритм Эрлиха - алгоритм порождающий все разбиения любого заданного числа в словарном порядке, использующий мультимножественную запись слагаемых, где учитывается их кратность. Более совершенен в вычислительном отношении чем алгоритм Гинденбурга.

Представление числа в виде суммы целых положительных слагаемых

Задача: перечислить все представления целого положительного числа n в виде суммы последовательности невозрастающих целых положительных слагаемых в лексикографическом порядке.

Например, для $n=5$ получим

1+1+1+1+1, 2+1+1+1, 2+2+1, 3+1+1, 3+2, 4+1, 5

Решение рекурсивное: будем использовать массив слагаемых A , состоящий из n элементов, n -максимальное количество слагаемых (все слагаемые – единицы).

Пусть $a[1], a[2], a[3], \dots, a[\text{last}]$ – невозрастающая последовательность целых чисел, сумма которых $s \leq n$; last – индекс последнего взятого элемента.

Если сумма стала равна заданному числу n , то выводим последовательность чисел.

Иначе подбираем следующее слагаемое j , начиная с единицы, которое не больше последнего слагаемого $j \leq a[\text{last}]$ и которое не увеличит сумму больше n , т.е. $j \leq (n-s)$.

Взяв очередное слагаемое, продолжаем комбинировать последовательность.

Представление числа в виде суммы целых положительных слагаемых

Решение нерекурсивное: будем использовать массив слагаемых A , состоящий из n элементов, n - максимальное количество слагаемых (в случае, когда все слагаемые – единицы).

В начале алгоритма генерации массив разбиений и количество слагаемых задаем

$$a[1] = a[2] = \dots = a[n] = 1,$$
$$\text{last} = n$$

В конце они примут значения

$$a[1] = n$$
$$\text{last} = 1$$

После задания начальных значений вызываем функцию генерации следующего разбиения, на вход подаем указатель (ссылку) на массив a и значение last

Представление числа в виде суммы целых положительных слагаемых

- В каком случае можно увеличить $a[k]$, не меняя предыдущих?
 - 1) Должно быть $a[k-1] > a[k]$ или $k=1$
 - 2) k должно быть не последним элементом (увеличение k надо компенсировать уменьшением следующих).
 - Увеличив $a[k]$, все следующие элементы надо взять минимально возможными.
 - Примеры: Пусть $N = 10$. Зададим разбиения и сгенерируем 3 следующих
 - 22222 $\rightarrow$ 31111111 $\rightarrow$ 3211111 $\rightarrow$ 322111 $\rightarrow$ 32221 $\rightarrow$ 331111 $\rightarrow$ 33211 $\rightarrow$ 3322 $\rightarrow$
 - 3331 $\rightarrow$ 4111111 $\rightarrow$ 421111 $\rightarrow$ 42211 $\rightarrow$ 4222 $\rightarrow$ 43111 $\rightarrow$ и т.д.
1. Просматриваем элементы разбиения справа налево, начиная с предпоследнего ($last-1$), до тех пор, пока не встретим $a[k-1] > a[k]$ или до левой границы массива $k=1$.
 - 2. Элемент $a[k]$ увеличиваем на единицу
 - 3. Вычисляем sum - сумму элементов, находящихся справа от k -го
 4. Начиная с $a[k+1]$ -го элемента присваиваем единицы ($sum-1$) элементам
 5. Вычисляем количество слагаемых в полученном разбиении $last = k + sum - 1$
 6. Вывод элементов массива a с $a[1]$ до $a[last]$

Представление числа в виде суммы целых положительных слагаемых

```
• // функция генерации следующего разбиения
• 01  k = last - 1
• 02  while (k≠1) && (a[k-1] <= a[k]) do k--
•      // k-е слагаемое увеличиваем
• 03  a[k]++
• 04  sum = 0    // сумма членов, стоявших после a[k]
• 05  for i = k+1 to last do
• 06      sum = sum + a[i]
• 07  for i = 1 to sum - 1 do
• 08      a[k + i] = 1
• 09  last = k + sum - 1;
• 10  for i = 1 to last do
• 11      вывод a[i]
```

Алгоритм генерации композиции по номеру

```
Unrank(num,n,m) begin
    if n < m return;
    if n=m
    then v[1]:=m
    else begin
        if(num<c (n - m)) //переход по правой ветви дерева, операция pos_m(v)
            begin Unrank(num,m,n-m);
                pos_m(v);
            end else begin //переход по левой ветви дерева, операция inc(v)
                num:=num-cm(n - m);
                Unrank(num,m,n-1);
                inc(v);
            end
        end
    end
end
```

Алгоритм генерации номера композиции

```
algorithm Rank(v,m,n) begin
    if m=n then return 0;
    if [1]=m then
        v.erase(v.begin())
        return Rank(v,m,n-m)
    else begin
        v[0]:= v[0]-1
        return Rank(v,m,n-1)+cm(n - m)
    end
end
```


Подмножества

- **Задача:** для заданных N и k ($k \leq N$) перечислить все k -элементные подмножества множества $\{1 \dots N\}$
- **Решение** рекурсивное: будем представлять каждое подмножество массивом A , длины N , состоящим из 0 и 1, в котором ровно k единиц.
- Последовательности упорядочим лексикографически.
- В каком случае s -й элемент последовательности можно увеличить, не меняя предыдущие?
- Если $a[s]$ меняется с 0 на 1, то для сохранения общего числа единиц надо справа от $a[s]$ заменить 1 на 0.
- Для этого надо, чтобы справа от $a[s]$ единицы были.
- $a[s]$ должен быть первым справа нулем, за которым стоят единицы. Т.е. $a[s+1]=1$.
- Надо искать наибольшее s , для которого $a[s] = 0$ и $a[s+1]=1$.
- Заменяв $a[s]$ на единицу (и $a[s+1]$ на 0?), надо записать следующие элементы в порядке возрастания – сначала нули, затем единицы.

Генерация k элементных подмножеств

- Начиная справа ищем $a[s]$, это должен быть первым справа нулем, за которым стоят единицы. Т.е. $a[s]=0$, $a[s+1]=1$. Т.е. надо искать наибольшее s , для которого $a[s] = 0$ и $a[s+1]=1$.
- Заменяя $a[s]$ на единицу (и $a[s+1]$ на 0?), надо записать следующие элементы в порядке возрастания – сначала нули, затем единицы.
- Можно при просмотре справа налево посчитать количество нулей и потом записать на один больше, после чего записать единицы до конца массива.

Пример

- Пусть $n=5$, $k=3$.
- Массив из 5 элементов должен содержать 3 единицы и 2 нуля.
- Запишем в лексикографическом порядке.

00111	10101
01011	10110
01101	11001
01110	11010
10011	11100

ИСТОЧНИКИ

1. Скиена, С. С. Алгоритмы. Руководство по разработке. БХВ-Петербург, 2011. (Стр. 251-292, 440, 455-492)
2. Макконнелл, Д. Д. Анализ алгоритмов. Активный обучающий подход. Техносфера, 2009. - 415 с. (стр. 67 – 69)
3. Окулов С. М. Программирование в алгоритмах / 3-е изд. – М. :БИНОМ. Лаборатория знаний, 2007. – 383 с. (стр. 27-76)

